DOSSIER TECHNIQUE ET FONCTIONNEL

PROSYS — ODOO 14

Rapport d’architecture, de fonctionnement et d’exploitation

3 novembre 2026

27 figures graphiques • 12 schémas textuels • Explication du fonctionnement applicatif et de l’infrastructure

Odoo 14 • Debian 12 • LXC • Python 3.8 / venv • PostgreSQL 15 • Patroni • etcd • HAProxy • GlusterFS

Configuration de référence : pg_ha_enabled = true

Périmètre métier : ensemble des familles ERP demandé par le porteur du projet.

Rapport fondé sur le dépôt local ; les états historiques sont identifiés. Les schémas sont intégrés dans ce document.

Version 1.0 — Projet ProSys

Sommaire

1. Objet et lecture du rapport

2. Ce que fait réellement l’application

3. Architecture actuelle et responsabilités

4. Installation Debian, Odoo et venv

5. Haute disponibilité PostgreSQL

6. Sauvegarde, restauration et exploitation

7. Atlas des schémas actuels

8. Atlas historique du projet

9. Schémas textuels de la documentation

10. Glossaire — application et infrastructure

Les liens du sommaire et les titres du volet de navigation donnent accès aux chapitres et aux figures.

1. Objet et lecture du rapport

Ce rapport explique le service Odoo du projet ProSys, son hébergement et les mécanismes qui rendent les données accessibles aux trois instances. Il rassemble les 27 vues graphiques distinctes du dépôt et les 12 schémas textuels d'architecture présents dans sa documentation.

La référence technique est constituée par l'inventaire, les variables et les tâches Ansible exécutables au 3 novembre 2026. Le mode PostgreSQL haute disponibilité est sélectionné par pg_ha_enabled: true. Les anciennes figures sont conservées avec leur contexte : elles racontent l'évolution du projet et ne décrivent pas toutes simultanément l'installation actuelle.

Trois niveaux d'information

Niveau	Ce que signifie l'information
Vérifié dans le dépôt	Configuration, scripts, chemins et enchaînements effectivement présents dans les fichiers examinés.
Déclaré par l'utilisateur	Périmètre métier étendu : l'utilisateur a demandé de couvrir l'ensemble des fonctions ERP (« tout »).
À constater en exploitation	Modules effectivement installés, primaire élu, santé réelle des services, délais de reprise et réussite d'une restauration.

Comment utiliser le document

- Lire les chapitres 2 à 6 pour comprendre les usages, les données, l'installation et l'exploitation.
- Utiliser l'atlas actuel comme référence de topologie, puis l'atlas historique pour comprendre les changements.
- Consulter les sources et le glossaire en fin de rapport pour retrouver les fichiers et les termes techniques.

Le rapport analyse les fichiers locaux. Aucun serveur n'a été interrogé pour déterminer les modules actifs, le primaire courant ou les performances. Les mots de passe et les clés privées ne sont pas reproduits.

Sources principales : [S01] inventaire ; [S02] variables communes ; [S03] playbook de déploiement.

2. Ce que fait réellement l’application

Odoo 14 est le service de gestion accessible depuis un navigateur. Trois instances applicatives utilisent une même base PostgreSQL, nommée prod par défaut, et un même stockage de fichiers. Elles constituent trois points d’accès à une application commune : une information validée en base peut être relue depuis une autre instance.

Odoo prend en charge les écrans, les utilisateurs, les droits d’accès, les règles métier des modules installés et les traitements planifiés. PostgreSQL conserve les données structurées ; GlusterFS fournit les fichiers communs. Le présent dépôt automatise surtout la mise en place et l’exploitation de cet ensemble.

Le dépôt et le produit Odoo ont des rôles différents

Élément	Rôle réel
Odoo 14	Exécute les modules et répond aux opérations des utilisateurs.
Dépôt ProSys	Déploie les conteneurs, le réseau, Odoo, Python, la base, les certificats, le stockage et des parcours de maintenance.
Traitement spécifique présent	Migration technique des pièces jointes entre base et filestore, avec contrôles des références et fichiers.
Développement métier propre	Aucun addon personnalisé, manifeste d’addon, modèle ou contrôleur métier identifié dans ce dépôt.

Périmètre métier retenu

À votre demande de couvrir « tout », les pages suivantes présentent les principales familles ERP : relation client, ventes, achats, stocks, finance, RH, projets et fonctions complémentaires. Il s’agit du périmètre métier déclaré et des capacités des applications Odoo correspondantes, pas d’un inventaire technique des modules de la base prod.

L’initialisation prouvée dans le code installe le module base avec --without-demo=all. Elle ne suffit pas à établir l’activation de chaque application métier ni l’édition Community ou Enterprise utilisée.

Sources : [S04] installation Odoo ; [S05] initialisation de la base ; [S06] configuration applicative. Périmètre métier : réponse utilisateur du 3 novembre 2026.

2.1 Fonctions métier : gestion commerciale et finance

Le périmètre demandé couvre l'ensemble des familles fonctionnelles d'Odoo. Le tableau décrit leur rôle lorsqu'elles sont installées et configurées. L'édition et l'activation réelles restent à constater dans l'application ; le dépôt ne contient pas cet inventaire.

Famille	Ce que l'utilisateur réalise	Résultat suivi
CRM	Enregistre des pistes et opportunités, organise les étapes du pipeline et les prochaines actions commerciales.	Prospects, opportunités et activités
Ventes	Prépare les devis, confirme les commandes et relie les ventes à leur livraison et à leur facturation.	Devis et commandes clients
Point de vente	Saisit les paniers et encaissements, puis suit les sessions de caisse.	Ventes en boutique ou restauration
Achats	Prépare les demandes de prix et commandes fournisseurs, puis suit les approvisionnements.	Commandes et factures fournisseurs
Stocks / logistique	Enregistre les réceptions, transferts et livraisons ; suit emplacements, lots et numéros de série selon le paramétrage.	Mouvements et traçabilité des articles
Facturation / comptabilité	Émet ou traite les factures, enregistre les paiements et rapproche les mouvements bancaires avec les pièces correspondantes.	Créances, dettes et écritures

Exemple d'enchaînement métier

Une opportunité CRM peut conduire à un devis, puis une commande. Les applications configurées organisent ensuite la livraison, la facture et le paiement. Les équipes travaillent sur des objets reliés : la base commune évite de recopier séparément les mêmes informations sur les trois serveurs Odoo.

Ces liaisons dépendent des modules, des droits, des produits, des taxes et des règles configurés. Le rapport décrit un parcours fonctionnel, pas un test d'une opération réelle dans prod.

Références produit officielles Odoo 14 : [O01] CRM, [O02] ventes, [O03] point de vente, [O04] achats, [O05] stock, [O06] comptabilité.

2.2 Fonctions métier : équipes, production et services

Famille	Ce que l'utilisateur réalise
Ressources humaines	Tient les fiches salariés et les informations professionnelles ; organise les recrutements et les candidatures selon les applications présentes.
Projets et feuilles de temps	Organise les prestations en projets et tâches, saisit le temps consacré et peut l'utiliser pour la facturation.
Fabrication	Prépare les nomenclatures et ordres de fabrication, suit les composants et, selon les modules, les opérations de travail.
Maintenance	Répertorie les équipements et suit les interventions préventives ou correctives avec les équipes responsables.
Site web / eCommerce	Publie le site et le catalogue ; reçoit les commandes en ligne et les relie au parcours commercial.
Marketing	Prépare et cible les campagnes de courriels ; événements et enquêtes complètent ce périmètre selon les applications.
Assistance clients	Suit les demandes en tickets affectés à des équipes et peut relier le temps de support à la facturation.

Ce que la plateforme apporte à ces métiers

Toutes ces applications partagent l'infrastructure Odoo : utilisateurs, accès, traitement des requêtes, données SQL et pièces jointes. Le cluster réduit la dépendance à une seule instance applicative et prépare la reprise du service SQL. Il ne définit pas à lui seul les règles de gestion d'une entreprise.

Édition et paramétrage

Les fonctions disponibles varient avec l'édition, les modules installés et le paramétrage. La présence d'un paquet Odoo 14 ne prouve pas la disponibilité de toutes les fonctions Enterprise. Le dépôt examiné ne déclare pas de chemin Enterprise explicite. Les fonctions avancées, notamment paie, Helpdesk ou certaines fonctions comptables, ne sont donc pas présentées comme vérifiées actives.

Références officielles Odoo 14 : [O07] RH, [O08] temps et prestations, [O09] fabrication, [O10] eCommerce, [O11] marketing, [O12] assistance, [O13] maintenance, [O14] éditions et installation.

2.3 Parcours concret d’un utilisateur

Ouvrir une page et enregistrer une information

- 1. Le navigateur rejoint l’une des IP Odoo sur le port 8069. Le bus de notifications utilise 8072 lorsque le mode workers est actif.
- 2. Le service Odoo exécute la requête avec le Python du venv /opt/odoo14-venv. Il traite la session et applique les droits et règles des modules concernés.
- 3. psycopg2 et libpq ouvrent une connexion vers l’un des trois HAProxy du réseau PostgreSQL, sur 5000.
- 4. HAProxy transmet la connexion au PostgreSQL primaire courant sur 5432. Les trois proxies conduisent au même primaire.
- 5. La transaction est exécutée puis validée ou annulée. Odoo construit la réponse affichée au navigateur.

Exemple : ajouter une pièce jointe

Le fichier envoyé par l’utilisateur est associé à un enregistrement ir_attachment. La base conserve les métadonnées et la référence ; le contenu est stocké dans le filestore commun, normalement /var/lib/odoo/filestore/prod pour la base par défaut. Une autre instance retrouve le même enregistrement et le même fichier.

La base SQL et le filestore sont donc complémentaires. Restaurer seulement PostgreSQL peut laisser des références vers des fichiers absents ; restaurer seulement les fichiers ne restitue pas les enregistrements et leurs droits.

Deux notions de « primaire »

odoo1 est le nœud applicatif désigné pour l’initialisation et le cron. Le primaire PostgreSQL est un rôle élu par Patroni et peut se trouver sur odoodb1, odoodb2 ou odoodb3. Une bascule SQL ne déplace pas automatiquement les tâches cron d’odoo1.

Sources : [S02], [S05], [S06], [S07] variables Odoo/cron.

3. Architecture actuelle et responsabilités

L’inventaire déclare six hôtes LXC et six conteneurs : trois pour les applications et trois pour les données. Les hôtes et les conteneurs ont des IP distinctes. Les domaines de panne physiques, hyperviseurs et équipements réseau ne sont pas détaillés dans les fichiers.

Service	IP du LXC	Hôte LXC	Fonction
odoo1	192.168.36.161	192.168.36.21	Application, initialisation unique et cron
odoo2	192.168.36.162	192.168.36.22	Application
odoo3	192.168.36.163	192.168.36.23	Application
odoodb1	192.168.33.161	192.168.33.21	PostgreSQL + Patroni + etcd + HAProxy
odoodb2	192.168.33.162	192.168.33.22	PostgreSQL + Patroni + etcd + HAProxy
odoodb3	192.168.33.163	192.168.33.23	PostgreSQL + Patroni + etcd + HAProxy

Rôle des composants

Composant	Responsabilité
PostgreSQL 15	Conserve les données SQL et alimente les répliques par les WAL.
Patroni	Supervise son PostgreSQL local et coordonne les rôles du cluster.
etcd	Conserve le verrou du leader et la configuration commune ; majorité 2/3.
HAProxy	Sélectionne le primaire grâce à l’API Patroni et relaie les sessions TCP.
GlusterFS externe	Présente le même volume de fichiers aux trois applications.
Ansible	Installe, configure, vérifie et orchestre les opérations prévues par les rôles.

Sources : [S01], [S02], [S03], [S08] rôle PostgreSQL HA.

3.1 Réseau, données et sécurité

Chemin réseau

Les LXC utilisent IPVLAN L2 : ils conservent une IP propre et partagent la MAC de l'interface physique parente. Les cartes hôtes sont enp5s0 pour Odoo et enp1s0 pour PostgreSQL. Le NAT des hôtes est désactivé, mais un SNAT externe vers 192.168.33.254 est documenté entre les deux réseaux.

Flux	Port / protocole	Utilité
Navigateur → Odoo	8069 / HTTP	Interface applicative
Navigateur → bus Odoo	8072 / HTTP	Notifications et longpolling
Odoo → HAProxy	5000 / TCP + TLS SQL	Entrée SQL multi-hôtes
HAProxy → PostgreSQL ; WAL	5432 / TCP + TLS	Requêtes et réplication physique
HAProxy → Patroni	8008 / HTTP	Sonde GET /primary
Patroni → etcd ; membres etcd	2379 / HTTPS ; 2380 / HTTPS	Coordination et consensus
Ansible → hôtes / LXC	22 / SSH	Administration

Protection SQL et stockage

HAProxy ne termine pas le TLS PostgreSQL. Odoo utilise verify-full pour contrôler le serveur ; le serveur exige le mot de passe et le certificat client Odoo. Les certificats clients résident dans /etc/odoo/pg-ssl, hors du volume partagé. Etcd emploie une autorité de certification distincte.

Le volume externe sre est monté directement dans chaque LXC Odoo sur /var/lib/odoo. Il porte les pièces jointes et les sessions. Le code, le venv, la configuration et les journaux restent locaux. Le serveur 192.168.36.8 fournit la description du volume ; les clients rejoignent ensuite les serveurs de briques indiqués.

Sources : [S01], [S02], [S06], [S09] montage GlusterFS ; [S08] templates Patroni, etcd et HAProxy.

4. Installation Debian, Odoo et venv

Le parcours actif prépare le stockage et la base avant d'initialiser Odoo. Les conteneurs sont Debian 12 bookworm. Le projet installe Odoo 14 depuis le paquet du dépôt nightly, compile CPython 3.8.20 dans /opt/python3.8 et crée un environnement virtuel local dans /opt/odoo14-venv.

Étape	Résultat attendu
Préparer LXC, horloge et réseau	Conteneurs joignables et heure cohérente avant les installations.
Monter GlusterFS	/var/lib/odoo est réellement monté dans chaque application.
Préparer PostgreSQL HA	Trois membres db_cluster et des endpoints HAProxy utilisables.
Installer Python et Odoo	Code du paquet Debian et bibliothèques du venv disponibles.
Configurer les services	odoo.conf, certificats SQL et fragments systemd posés.
Aligner les identités	Compte Odoo : UID 105 et GID 112 sur les instances.
Initialiser une seule fois	odoo1 initialise conditionnellement base, services Odoo arrêtés.
Démarrer les trois Odoo	Services persistants ; attente de l'ouverture de 8069.

Comment le venv utilise le code du paquet

Le code reste sous /usr/lib/python3/dist-packages/odoo. Le fichier odoo-deb.pth rend ce répertoire visible depuis le venv. Les bibliothèques requises sont installées dans le venv, avec un fichier figé s'il existe, sinon le requirements.txt amont. Le Python système 3.11 continue de servir au système et à Ansible.

```
/opt/python3.8/bin/python3.8 -m venv /opt/odoo14-venv
/opt/odoo14-venv/bin/python /usr/bin/odoo
Configuration : /etc/odoo/odoo.conf ; journal : /var/log/odoo/odoo.log
```

Sources : [S03], [S04], [S05], [S07], [S10] rôle python38.

4.1 Démarrage et traitements techniques

Pourquoi Odoo attend son stockage

systemd établit la dépendance de montage avec RequiresMountsFor=/var/lib/odoo. AssertPathIsMountPoint exige ensuite un véritable point de montage. Le rôle Ansible vérifie également le type GlusterFS. Ces garde-fous évitent qu'Odoo écrive sans bruit sur le disque local si le volume partagé manque.

venv.conf remplace la commande de démarrage par le Python du venv et fournit les variables TLS PostgreSQL. L'enveloppe /usr/local/bin/odoo utilise le même interpréteur pour les commandes. Chaque instance possède deux workers ; le cron est configuré à 1 sur odoo1 et 0 sur les deux autres.

Migration des pièces jointes réellement présente

ansi_syncro.yaml orchestre une migration du support de stockage, et non une réplication continue entre applications. Le parcours sauvegarde, arrête les Odoo, récupère les anciens filestores, déplace les pièces jointes entre base et disque avec odoo shell, vérifie le résultat et redémarre selon ses tâches.

Le traitement utilise le modèle standard ir.attachment, inclut les champs binaires res_field et isole les traitements par savepoint. Il contrôle les marqueurs ECHECS=0, RESTANT=0 et SANS_FICHIER=0. Une vérification compare aussi store_fname aux fichiers présents et distingue les absents des orphelins.

Procédures historiques à réexaminer en mode HA

Certains parcours de migration visent encore le groupe statique db_containers, qui peut ne plus désigner le primaire après une bascule. Le nettoyage APT arrête Odoo ; le reset complet touche /var/lib/odoo, aujourd'hui partagé. Ces routines ne doivent pas être assimilées à des opérations sans interruption sur la nouvelle architecture.

Sources : [S04], [S07], [S09], [S11] migration des pièces jointes ; [S12] nettoyage.

5. Haute disponibilité PostgreSQL

Le mode actuel sélectionne Patroni. Chaque nœud de données héberge PostgreSQL, Patroni, etcd et HAProxy. Un seul PostgreSQL reçoit les connexions applicatives ; les deux autres reçoivent directement ses journaux WAL. Patroni ne transporte pas ces journaux.

Bascule automatique

Si le primaire devient indisponible, la coordination peut promouvoir une réplique éligible à condition que la majorité etcd soit disponible. HAProxy retire une cible invalide, ferme ses sessions et oriente les nouvelles connexions vers le nouveau primaire identifié par /primary. Les transactions interrompues ne sont pas transférées.

Paramètre ou mécanisme	Valeur / portée
Quorum etcd	2 membres sur 3 ; consensus de coordination, distinct des acquittements SQL
Réplication	Asynchrone : perte de transactions récentes possible après promotion
ttl / loop_wait / retry_timeout	30 s / 10 s / 10 s ; ne constituent pas un délai de reprise mesuré
maximum_lag_on_failover	1 048 576 octets : seuil d’éligibilité, pas garantie de perte maximale
Sondes HAProxy	inter 3 s, fall 3, rise 2 ; /primary sur HTTP/8008
Watchdog matériel	Désactivé dans les LXC
Retour d’un ancien primaire	pg_rewind si ses préconditions sont satisfaites ; sinon diagnostic et éventuelle réinitialisation explicite

En partition réseau 2+1, le côté etcd majoritaire conserve la capacité de décision. Un Patroni privé du renouvellement du verrou tente l’arrêt ou la démotion de son PostgreSQL. La perte du quorum empêche la coordination normale ; un membre isolé ne remplace pas la majorité.

Sources : [S08]. Références : API Patroni, modes de réplication, watchdog et pg_rewind PostgreSQL 15 ; voir [R01–R04].

5.1 Maintenance et relance récente de Patroni

Le rôle a récemment ajouté un parcours pour corriger les endpoints etcd réellement chargés par Patroni. Il compare la section etcd3 du fichier /etc/patroni/config.yml avec la copie prise au démarrage dans /run/patroni/config.yml. Le rapport complète ainsi les premiers schémas, qui ne montrent pas ce mécanisme.

- 1. Identifier les agents dont la configuration chargée est périmée ou inconnue et déterminer le primaire courant.
- 2. Conserver une pause de maintenance déjà présente, ou poser une pause du cluster.
- 3. Exiger la preuve que chacun des trois agents voit la pause avant toute relance.
- 4. Relancer les agents des répliques un à un, puis celui du primaire en dernier.
- 5. Vérifier le retour de Patroni, son accès au DCS et la conservation de l’heure de démarrage de PostgreSQL.
- 6. Retirer uniquement la pause posée par ce passage ; conserver une pause préexistante.

Conséquence opérationnelle de la pause

Un échec peut laisser le cluster en pause. Dans cet état, pg_ha_enabled reste true, mais la bascule automatique est suspendue. La présence de cette variable ne suffit donc pas à prouver la reprise automatique effective.

Le parcours vise à relancer les agents sans redémarrer PostgreSQL et en vérifie les conditions. Il ne constitue pas une garantie générale d’absence d’interruption côté client. Les sondes HAProxy et les connexions doivent toujours être surveillées.

Contrôles de fin de rôle

Les vérifications actuelles interrogent les endpoints etcd avec les certificats Patroni, contrôlent la lecture du cluster par patronictl et la fraîcheur du contact DCS si disponible, puis les rôles, le retard des répliques et le chemin SQL par chaque HAProxy. La remise en conformité du DCS conserve une pause existante.

Sources : [S08] tasks/relance.yaml, files/relancer_patroni.py, tasks/configuration.yaml, tasks/verif.yaml et template patroni.service.j2.

6. Sauvegarde, restauration et exploitation

L’hôte 192.168.36.21 conserve un montage GlusterFS en lecture seule sur /mnt/odoo. Le rôle de sauvegarde prévoit un dump SQL au format custom, puis une archive du volume excluant sessions et _sessions. Il vérifie la lisibilité des archives et produit un manifeste SHA256.

Élément	Configuration prévue
Destination	/srv/sauvegardes
Planification	02:15 + délai aléatoire maximal de 10 minutes
Rétention	30 jours
Copie vers le contrôleur	Optionnelle, désactivée par défaut
Contenu nécessaire	Base SQL et filestore cohérents
Validation complète	Une restauration applicative reste nécessaire ; la lisibilité des archives seule ne suffit pas

Écart identifié dans le parcours HA

Le rôle prépare les endpoints HA et .pgpass, mais ne déploie pas le certificat client SQL ni les variables PGSSL* exigés sur le chemin via HAProxy. Cette partie doit être complétée et validée avant de considérer les sauvegardes HA comme opérationnelles. Le playbook ansi_sauvegarde.yaml cité dans d’anciens documents n’est pas présent dans l’arborescence examinée.

Parcours de restauration expliqué

- Suspendre les écritures Odoo et choisir un dump et un filestore cohérents.
- Restaurer la base via le primaire courant et les fichiers par le point de montage du volume.
- Vérifier propriétaires, références des pièces jointes et accès applicatifs avant reprise.

Le dépôt ne fournit pas une restauration HA automatisée et validée. Les anciennes procédures mentionnant les réseaux .96.x ou /mnt/odoo14 ne doivent pas être reprises sans adaptation.

Sources : [S13] rôle et variables de sauvegarde ; [S08] documentation Patroni du projet.

6.1 Ce que la disponibilité couvre réellement

Situation	Effet et limite
Une application Odoo tombe	Les autres instances accèdent à la même base et aux mêmes fichiers, mais aucune adresse web unique ni bascule automatique du navigateur n'est établie.
Le primaire PostgreSQL tombe	Promotion possible sous conditions ; sessions et transactions en cours peuvent échouer.
odoo1 tombe	Le rôle cron n'est pas déplacé par l'élection PostgreSQL.
Le quorum etcd est perdu	La coordination normale du cluster n'est plus disponible.
GlusterFS est indisponible	Les pièces jointes et sessions communes sont affectées ; les garde-fous cherchent à empêcher un démarrage sur un répertoire local.
Une corruption logique survient	La réplication n'est pas une sauvegarde ; une erreur peut se propager aux copies.

Vérifications d'exploitation à documenter

- Constater les modules Odoo installés, leur édition et les usages métier réellement configurés.
- Identifier le primaire par Patroni, vérifier le quorum, le retard des répliques et la présence éventuelle d'une pause.
- Contrôler les montages partagés et des pièces jointes depuis plusieurs instances.
- Compléter le TLS de sauvegarde et mesurer une restauration base + filestore.
- Tester une bascule et mesurer l'interruption, puis définir les objectifs de reprise de l'organisation.

Le projet contient un exemple Nginx, mais ses adresses historiques et l'absence de déploiement actif ne permettent pas d'en déduire un répartiteur web en production. HAProxy, dans le parcours actuel, route le SQL.

Sources : [S01–S03], [S06–S09], [S13] ; nginx/odoo-repartition.conf et docs/NGINX-REPARTITION-CHARGE.md.

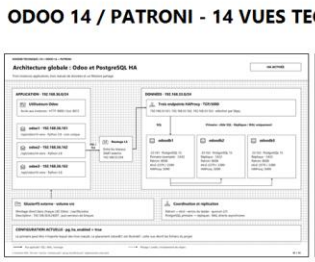
7. Atlas des schémas actuels

Référence principale : configuration du 3 novembre 2026

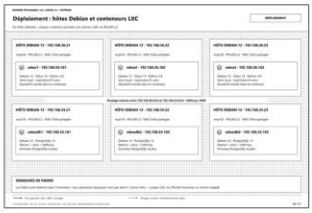
Les quatorze planches suivantes décrivent l'architecture HA sélectionnée, ses principaux scénarios et l'installation d'Odoo. Les exemples de primaire ne sont pas une observation du rôle réellement élu.

Chaque planche est accompagnée d'une lecture, d'un lien avec le fonctionnement applicatif et d'un point à retenir. Les grandes pages paysage conservent les schémas à une taille lisible ; les images sont intégrées au document.

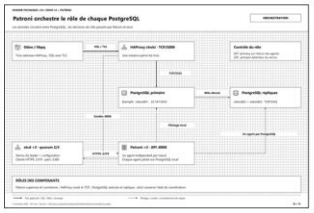
Le nouveau parcours de relance Patroni avec pause de maintenance est expliqué au chapitre 5.1 : il complète ces planches sans modifier leur topologie.



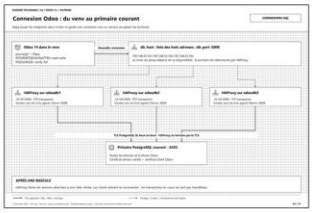
01 - Architecture globale : Odoo et PostgreSQL HA



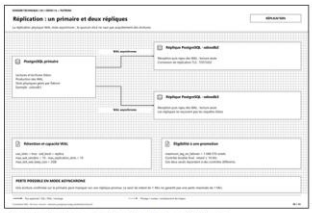
02 - Déploiement : hôtes Debian et conteneurs LXC



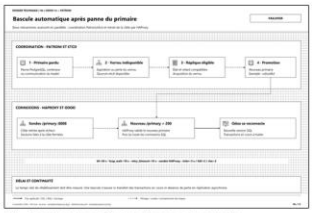
03 - Patroni orchestre le rôle de chaque PostgreSQL



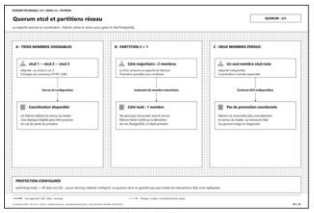
04 - Connexion Odoo : du venv au primaire courant



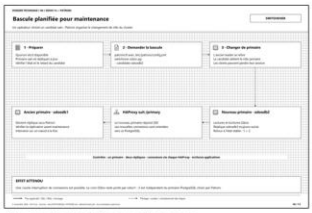
05 - Réplication : un primaire et deux répliques



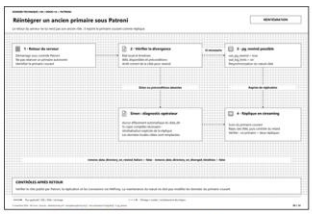
06 - Bascule automatique après panne du primaire



07 - Quorum etcd et partitions réseau



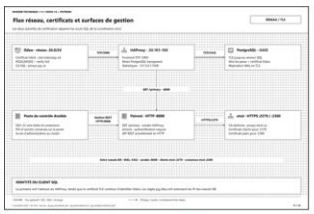
08 - Bascule planifiée pour maintenance



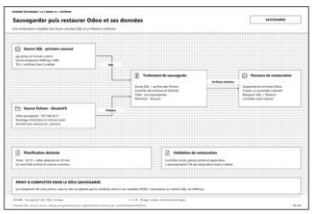
09 - Réintégrer un ancien primaire sous Patroni



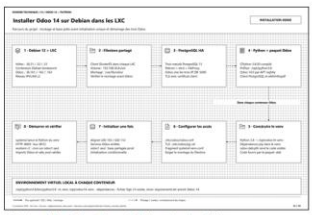
10 - Transition : du cluster initial au mode Patroni



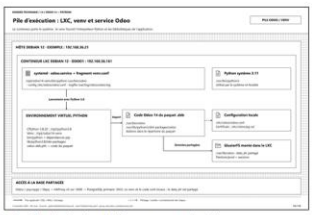
11 - Flux réseau, certificats et surfaces de gestion



12 - Sauvegarder puis restaurer Odoo et ses données



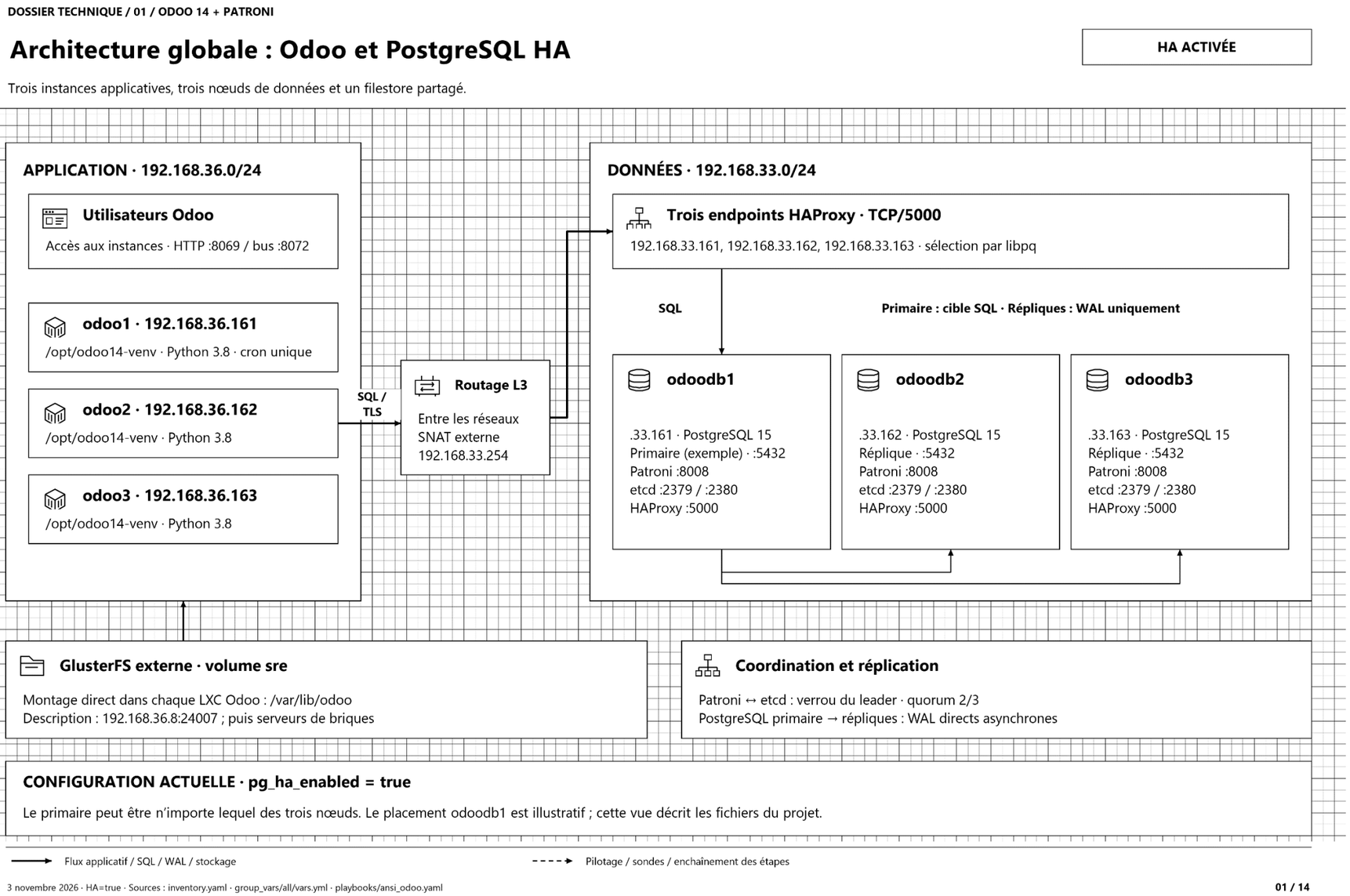
13 - Installer Odoo 14 sur Debian dans les LXC



14 - Pile d'exécution : LXC, venv et service Odoo

Figure 1 — Architecture globale : Odoo et PostgreSQL HA

ACTUEL — configuration du 3 novembre 2026 ; primaire illustratif



Lecture : Lire de gauche à droite : les utilisateurs rejoignent les applications Odoo ; leurs connexions SQL traversent le routage externe puis un HAProxy. Le proxy choisi transmet au primaire courant. Les WAL partent directement de ce primaire vers les deux répliques.

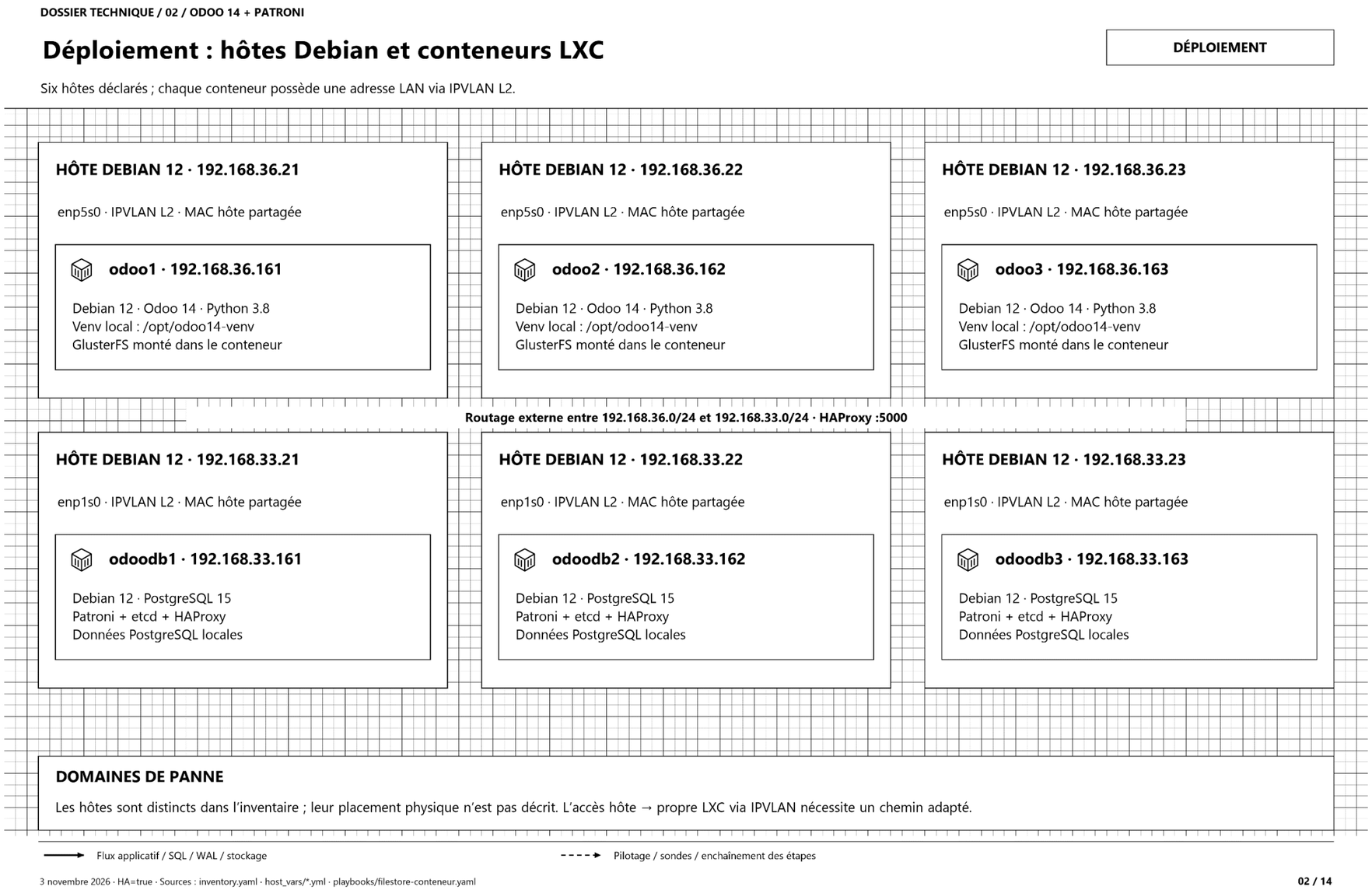
Fonctionnement : Les trois applications utilisent les mêmes données SQL et le même volume GlusterFS. Les quatre services des nœuds DB ont des rôles distincts : PostgreSQL stocke, Patroni supervise, etcd coordonne et HAProxy route.

À retenir : odoodb1 est un exemple de primaire. La redondance SQL ne fournit pas à elle seule une adresse web unique ou la disponibilité du stockage externe.

Source : inventory.yaml · group_vars/all/vars.yml · playbooks/ansi_odoo.yaml | Fichier : docs/schemas-patroni-odoo14/01-architecture-globale-ha.drawio

Figure 2 — Déploiement : hôtes Debian et conteneurs LXC

ACTUEL — configuration du 3 novembre 2026 ; primaire illustratif



Lecture : Chaque grand cadre représente un hôte et chaque cadre intérieur son conteneur. Les IP .21–.23 désignent les hôtes ; les IP .161–.163 désignent les LXC. Les applications et les bases occupent deux sous-réseaux distincts.

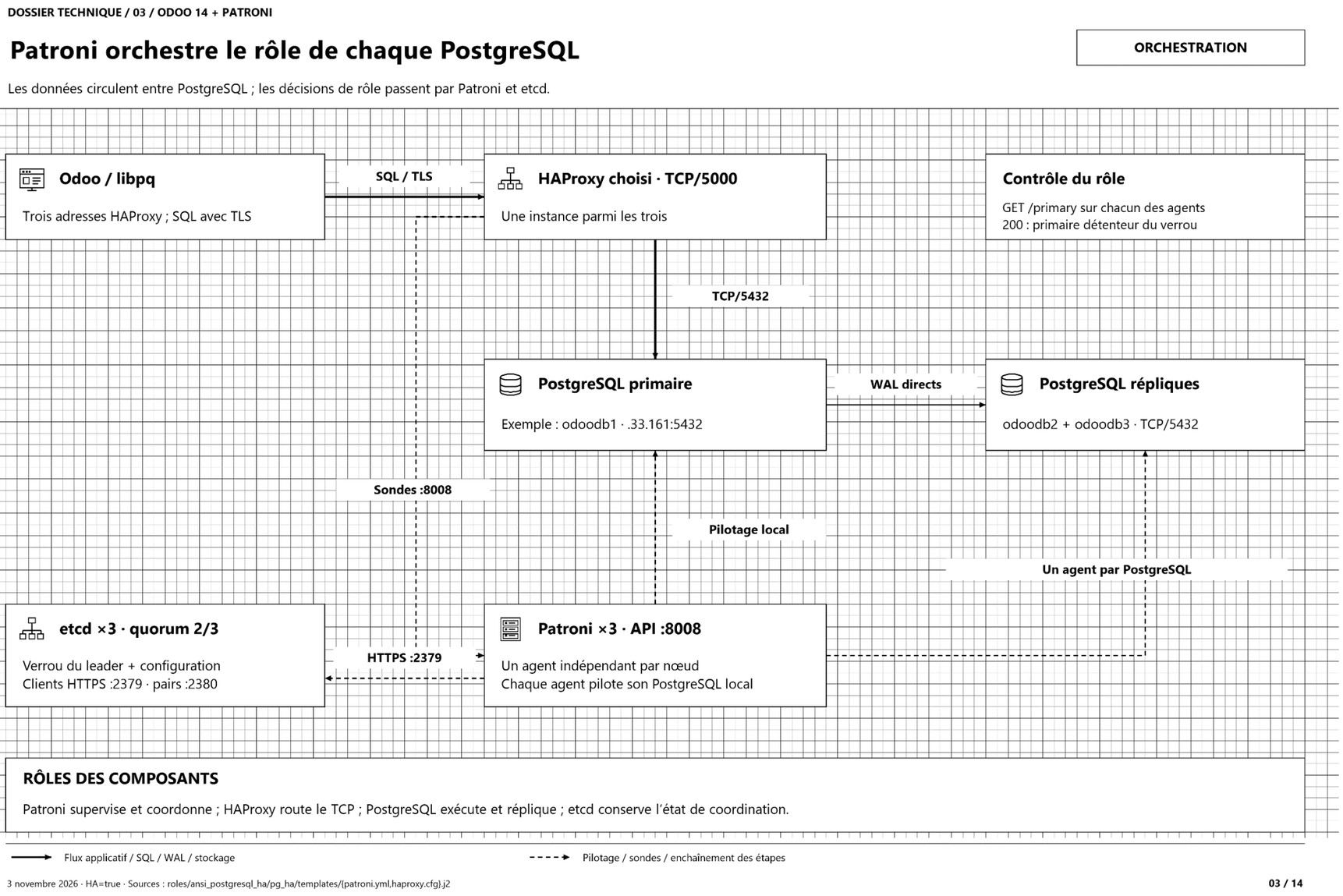
Fonctionnement : Cette séparation permet d'installer Odoo et PostgreSQL dans des environnements dédiés. IPVLAN L2 conserve la carte sur l'hôte et partage sa MAC avec le conteneur, tout en attribuant une IP propre au LXC.

À retenir : L'inventaire sépare les hôtes logiques ; il ne décrit pas leurs hyperviseurs ni des domaines physiques de panne indépendants.

Source : inventory.yaml · host_vars/*.yml · playbooks/filestore-conteneur.yaml | Fichier : docs/schemas-patroni-odoo14/02-deploiement-hotes-lxc.drawio

Figure 3 — Patroni orchestre le rôle de chaque PostgreSQL

ACTUEL — configuration du 3 novembre 2026 ; primaire illustratif



Lecture : Les traits SQL relient Odoo au proxy puis au primaire. Les WAL relient PostgreSQL à ses répliques. Les pointillés relient les sondes HAProxy aux agents Patroni et les agents à leur PostgreSQL local ou à etcd.

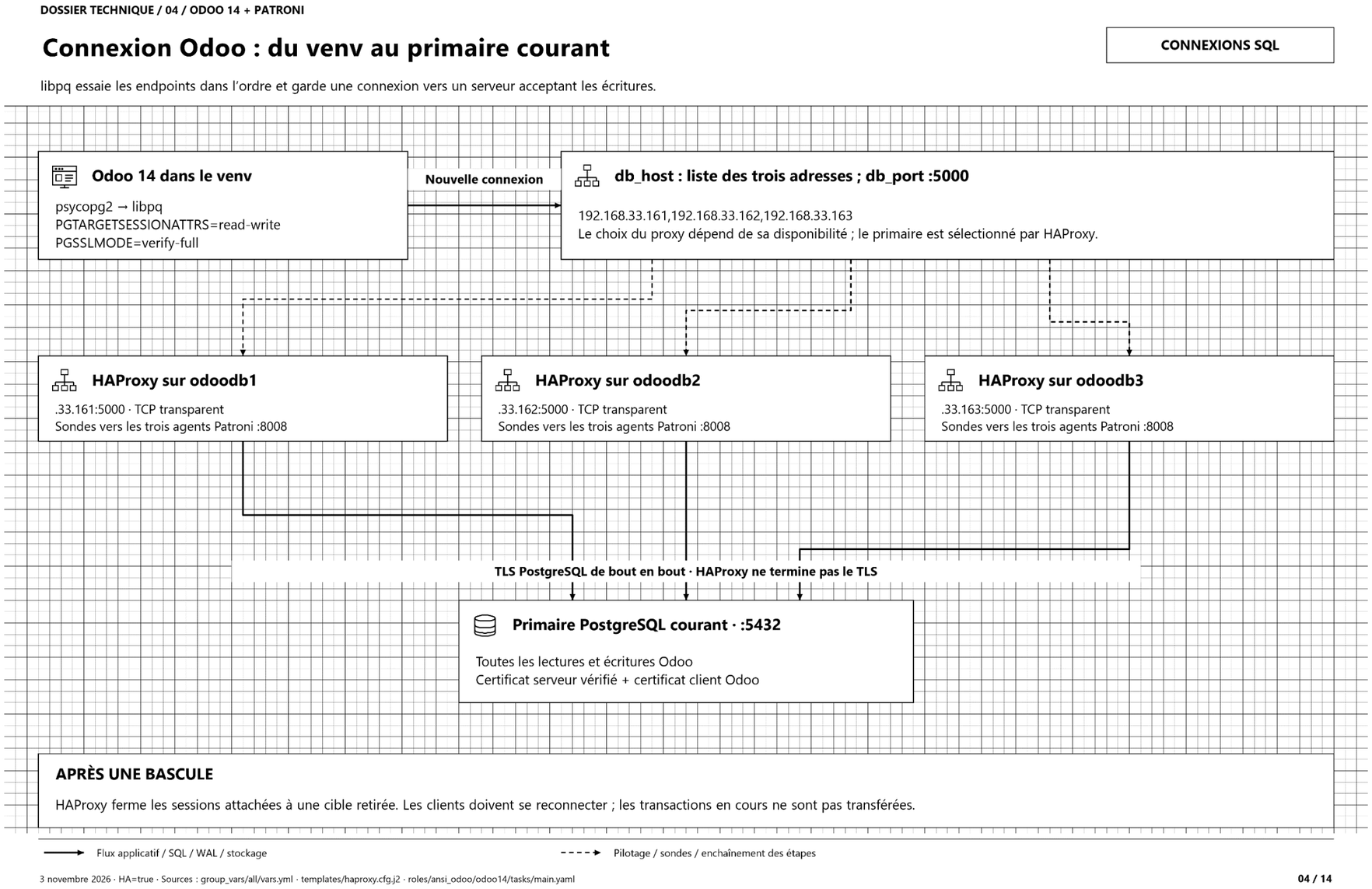
Fonctionnement : HAProxy consulte GET /primary. Patroni gère uniquement son serveur PostgreSQL local et participe à la coordination du leader. Etcd conserve le verrou et la configuration commune.

À retenir : Patroni ne transporte pas les données SQL ou WAL ; etcd n'est pas une copie de la base Odoo.

Source : roles/ansi_postgresql_ha/pg_ha/templates/{patroni.yml,haproxy.cfg}.j2 | Fichier : docs/schemas-patroni-odoo14/03-orchestration-patroni.drawio

Figure 4 — Connexion Odoo : du venv au primaire courant

ACTUEL — configuration du 3 novembre 2026 ; primaire illustratif



Lecture : Le client libpq essaie les trois adresses HAProxy sur 5000. Le proxy retenu relaie la session vers le primaire PostgreSQL sur 5432. Les trois entrées peuvent conduire au même serveur primaire.

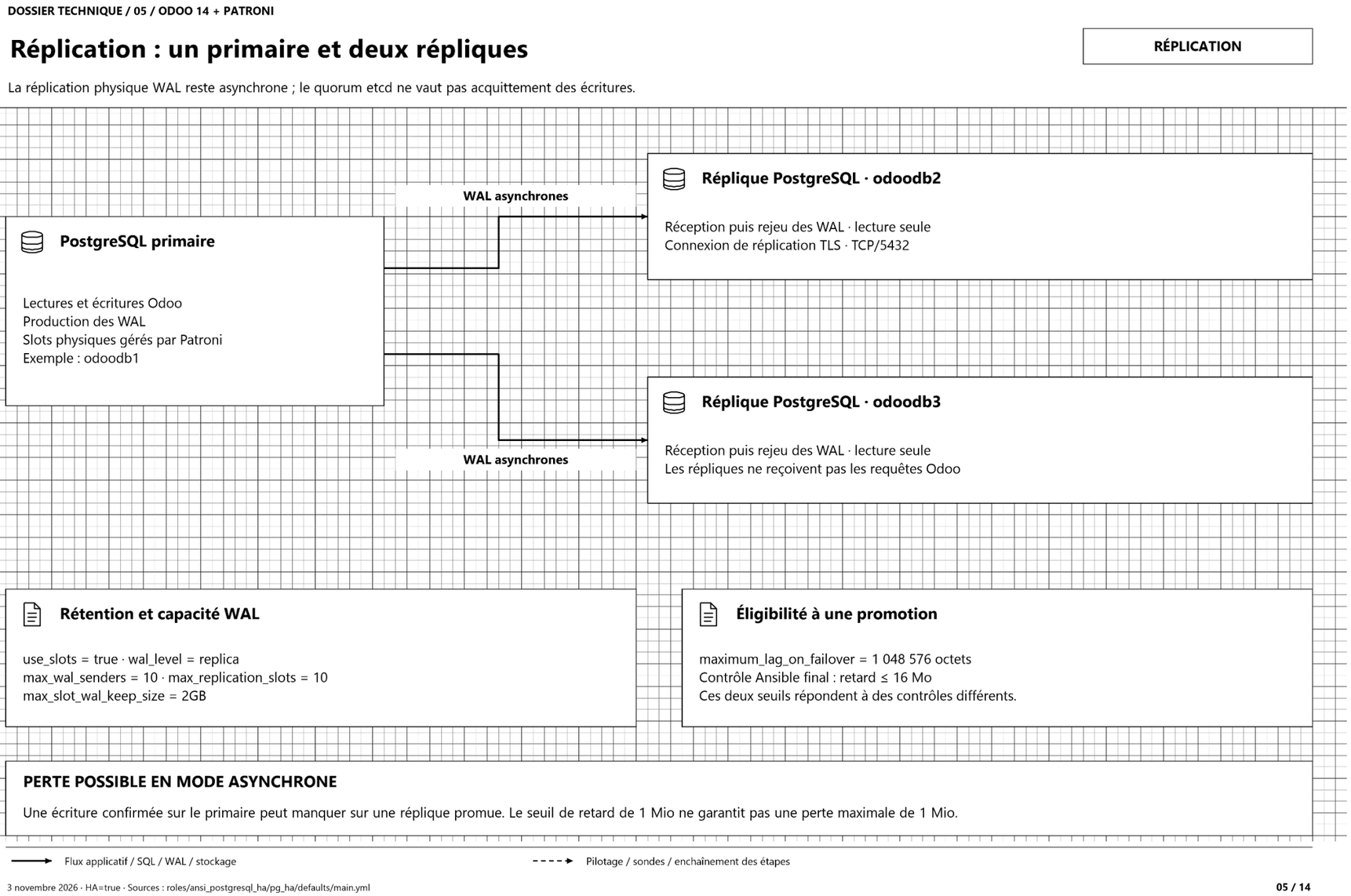
Fonctionnement : PGTARGETSESSIONATTRS=read-write permet d'écarter un endpoint aboutissant à un serveur en lecture seule. Le TLS traverse HAProxy sans y être terminé : le certificat serveur et le certificat client restent utilisés au niveau SQL.

À retenir : Le mécanisme porte sur une nouvelle connexion à la base ; il ne répartit pas les lectures sur les répliques et ne déplace pas une transaction en cours.

Source : group_vars/all/vars.yml · templates/haproxy.cfg.j2 · roles/ansi_odoo/odoo14/tasks/main.yaml | Fichier : docs/schemas-patroni-odoo14/04-connexion-odoo-haproxy.drawio

Figure 5 — Réplication : un primaire et deux répliques

ACTUEL — configuration du 3 novembre 2026 ; primaire illustratif



Lecture : Le primaire reçoit les écritures, produit les WAL et les transmet directement aux deux répliques. Celles-ci les reçoivent et les rejouent pour maintenir une copie physique en lecture seule.

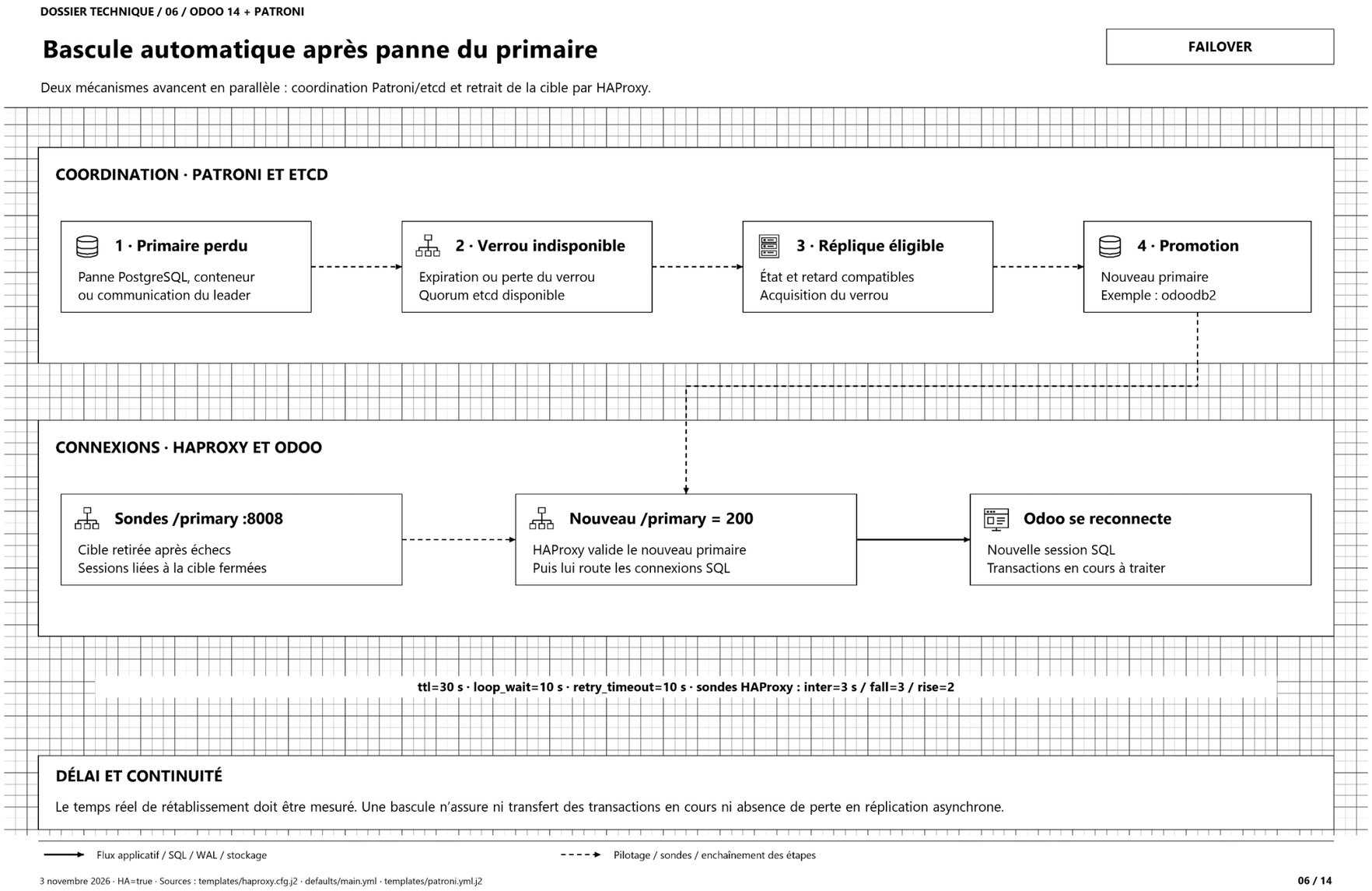
Fonctionnement : Patroni gère les rôles et les slots. La configuration prévoit wal_level=replica, des capacités de connexions de réplication et une limite de conservation WAL par slot.

À retenir : La réplication est asynchrone. Le seuil d'éligibilité à une promotion de 1 048 576 octets et le contrôle de retard final de 16 Mo ne garantissent pas une absence de perte.

Source : roles/ansi_postgresql_ha/pg_ha/defaults/main.yml | Fichier : docs/schemas-patroni-odoo14/05-replication-postgresql.drawio

Figure 6 — Bascule automatique après panne du primaire

ACTUEL — configuration du 3 novembre 2026 ; primaire illustratif



Lecture : Suivre les deux lignes : Patroni et etcd coordonnent la perte du verrou et la promotion ; HAProxy retire la cible invalide puis reconnaît le nouveau primaire. Le client ouvre ensuite une nouvelle session SQL.

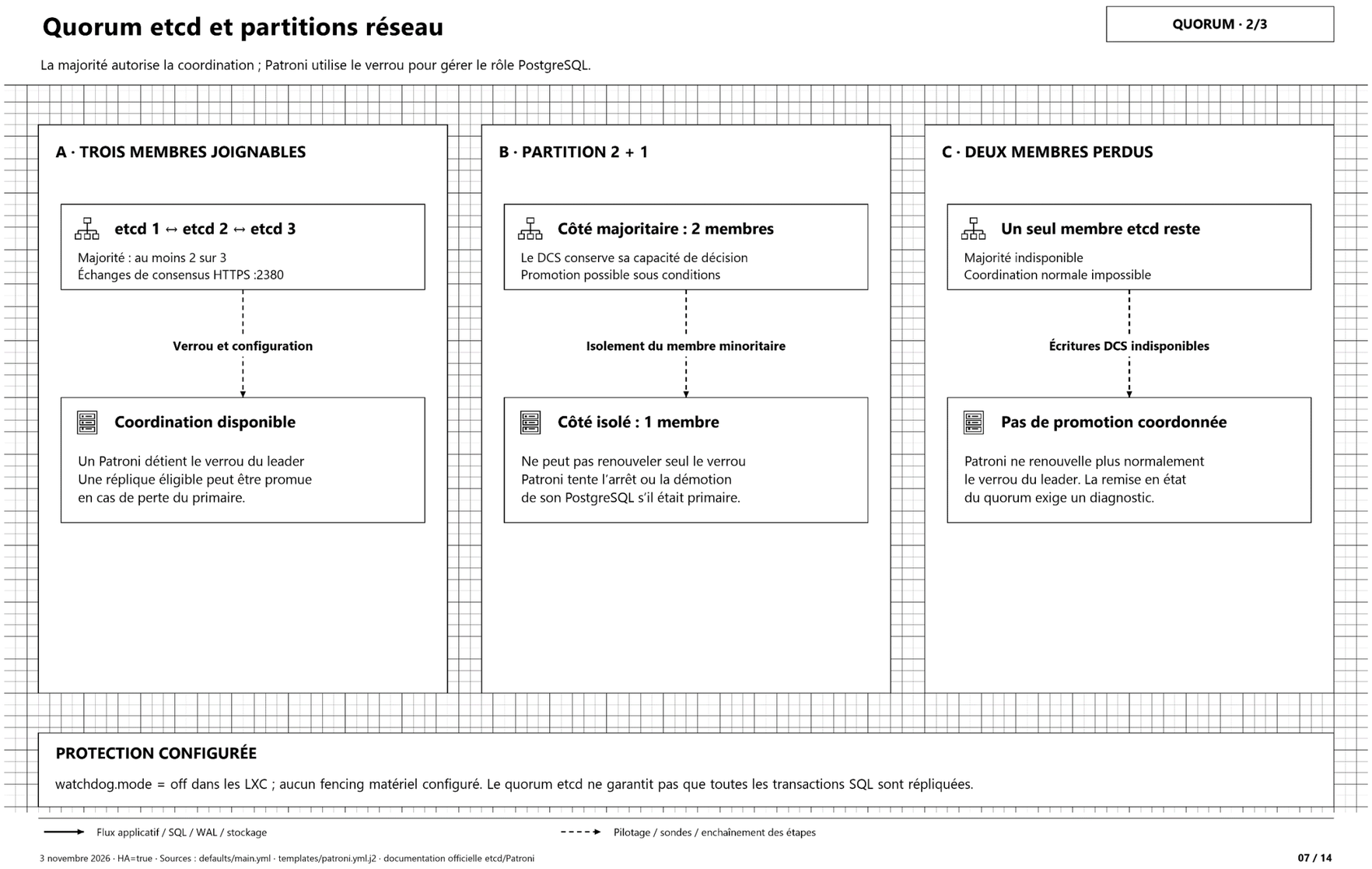
Fonctionnement : Une réplique saine et éligible peut être promue si le quorum de coordination subsiste. La fermeture des anciennes sessions évite de les conserver sur un serveur devenu inadapté aux écritures.

À retenir : Les opérations interrompues doivent être contrôlées avant répétition. Les paramètres de temporisation ne sont pas une mesure du temps réel de reprise.

Source : templates/haproxy.cfg.j2 · defaults/main.yml · templates/patroni.yml.j2 | Fichier : docs/schemas-patroni-odoo14/06-bascule-automatique.drawio

Figure 7 — Quorum etcd et partitions réseau

ACTUEL — configuration du 3 novembre 2026 ; primaire illustratif



Lecture : Comparer les trois situations : trois membres joignables, une partition 2+1, puis deux membres perdus. La majorité est de deux sur trois ; le membre isolé ne peut pas modifier seul l'état de coordination.

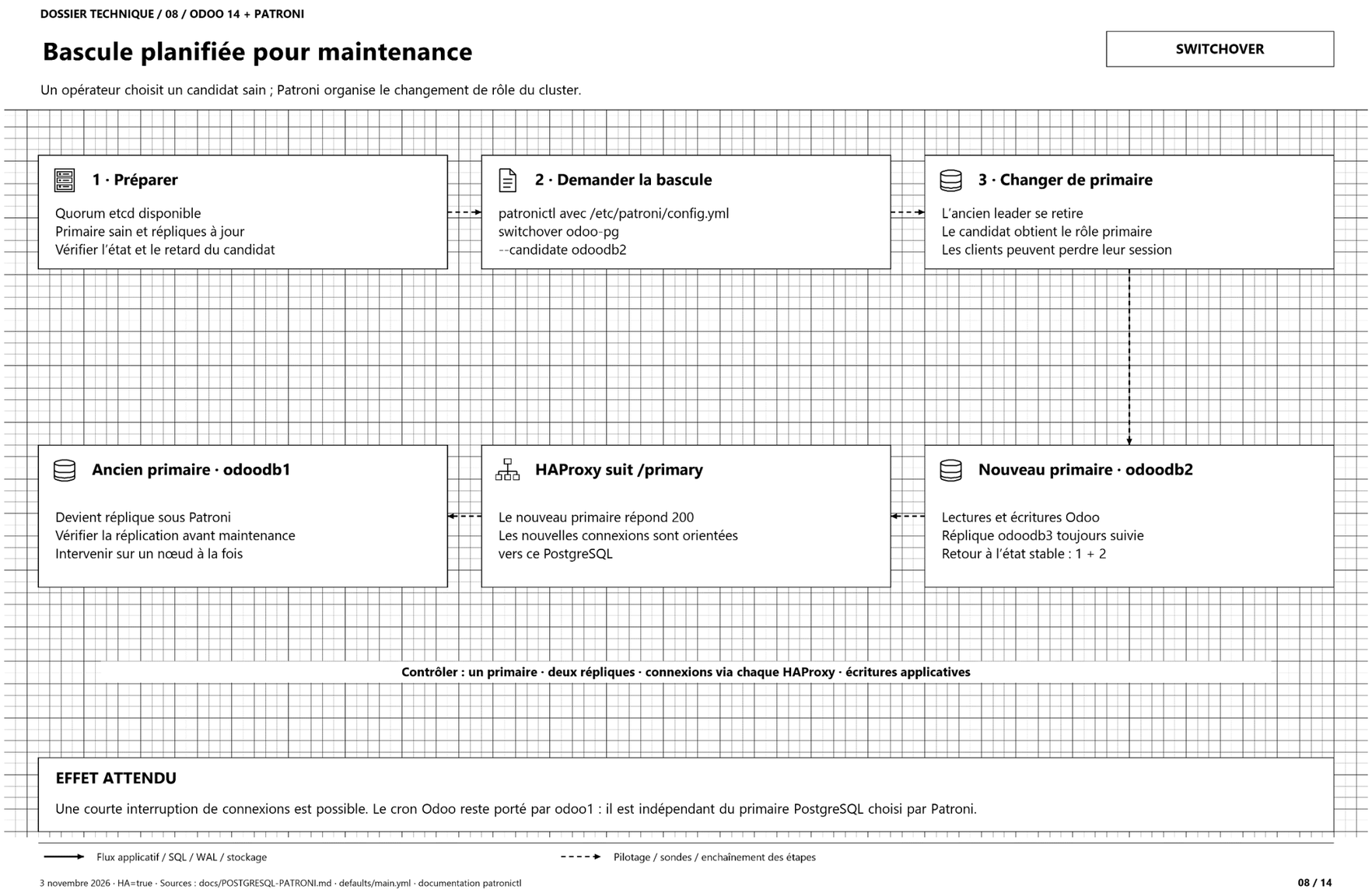
Fonctionnement : Patroni utilise cette coordination pour renouveler son verrou. Lorsqu'il ne peut plus le renouveler, il tente d'arrêter ou de démoter le PostgreSQL concerné.

À retenir : Le quorum etcd protège une décision de rôle. Il ne prouve pas que deux répliques PostgreSQL ont reçu toutes les transactions ; le watchdog matériel est désactivé.

Source : defaults/main.yml · templates/patroni.yml.j2 · documentation officielle etcd/Patroni | Fichier : docs/schemas-patroni-odoo14/07-quorum-et-partitions.drawio

Figure 8 — Bascule planifiée pour maintenance

ACTUEL — configuration du 3 novembre 2026 ; primaire illustratif



Lecture : Le parcours commence par un cluster sain et un candidat vérifié. L'opérateur demande une bascule ; Patroni change le leader, HAProxy suit le nouveau /primary et l'ancien primaire rejoint le cluster comme réplique.

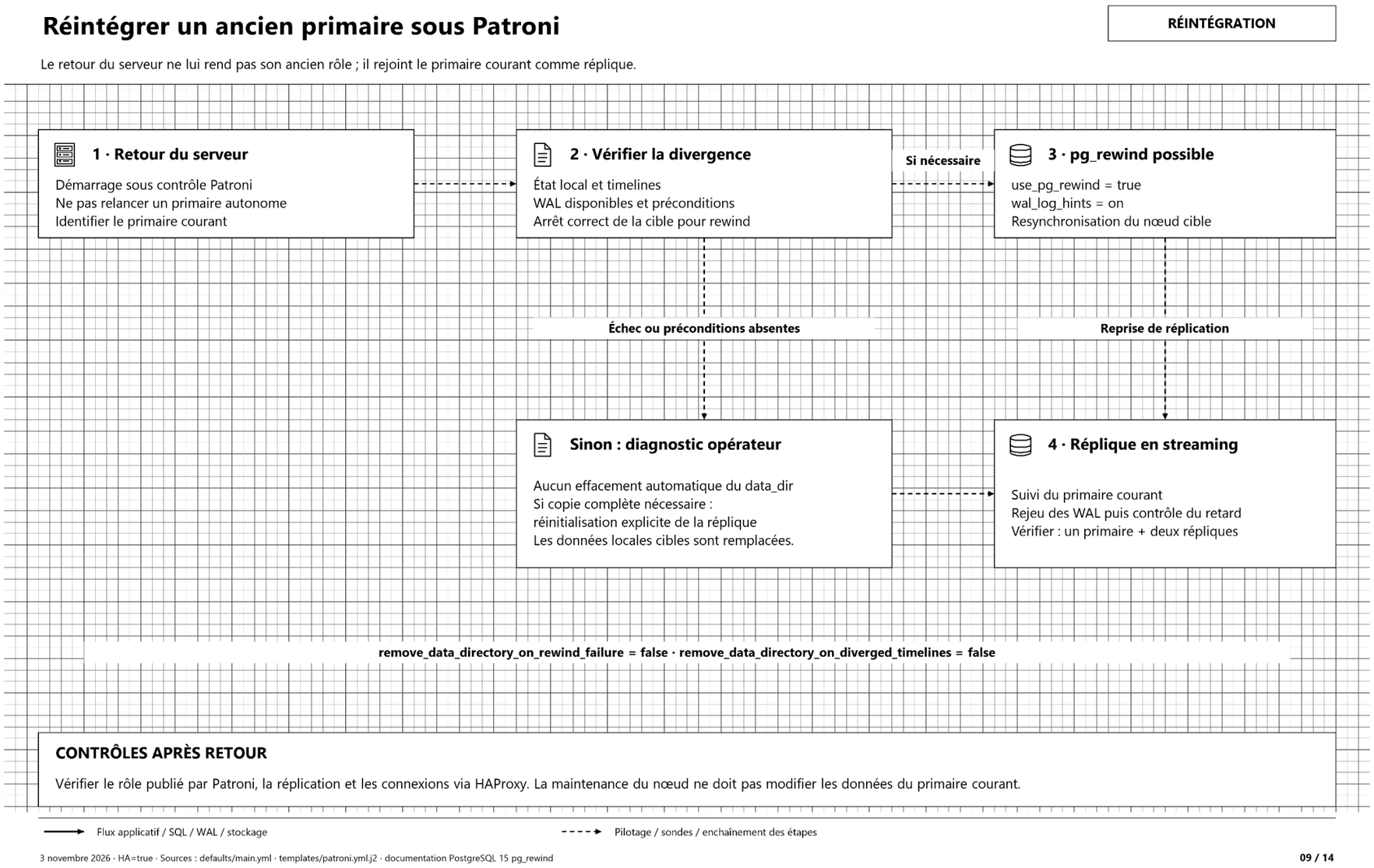
Fonctionnement : Ce scénario prépare la maintenance d'un serveur tout en conservant un primaire pour l'application. Les contrôles doivent vérifier le rôle, la réplication et les connexions via les différents proxies.

À retenir : Une interruption des connexions reste possible. Le cron applicatif d'odoo1 ne suit pas ce changement de primaire SQL.

Source : docs/POSTGRESQL-PATRONI.md · defaults/main.yml · documentation patronictl | Fichier : docs/schemas-patroni-odoo14/08-bascule-planifiee.drawio

Figure 9 — Réintégrer un ancien primaire sous Patroni

ACTUEL — configuration du 3 novembre 2026 ; primaire illustratif



Lecture : Au retour du serveur, Patroni examine l’état et la divergence des timelines. Si les préconditions sont réunies, pg_rewind prépare le retour en réplique, puis le rejeu WAL rétablit le suivi du primaire courant.

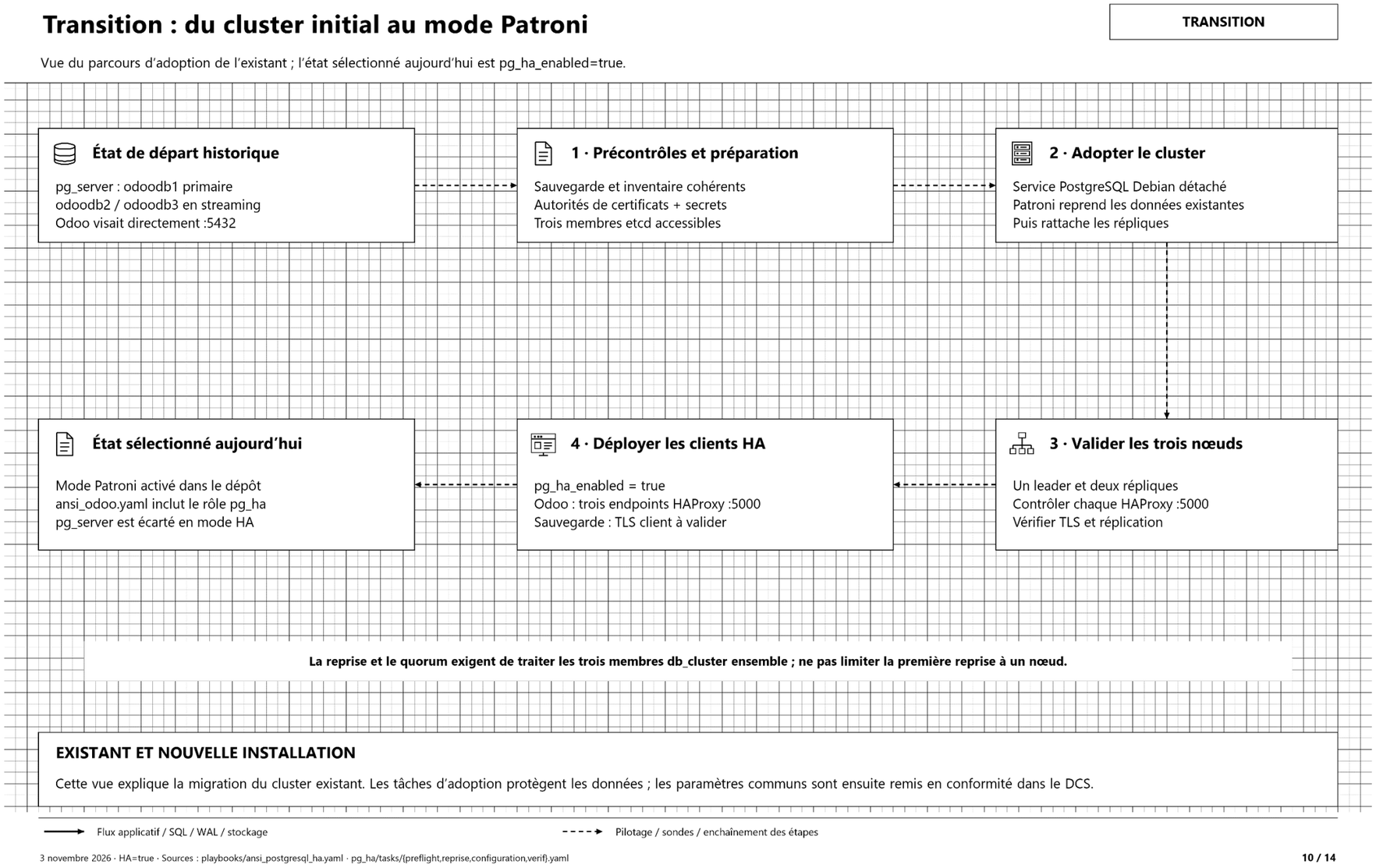
Fonctionnement : Le rôle autorise pg_rewind et active wal_log_hints. En cas d’échec ou de conditions absentes, le parcours demande un diagnostic avant une éventuelle réinitialisation complète.

À retenir : Le retour du serveur ne lui rend pas son ancien rôle. Aucun effacement automatique n’est configuré sur échec de rewind ; une réinitialisation explicite remplace les données locales de la réplique.

Source : defaults/main.yml · templates/patroni.yml.j2 · documentation PostgreSQL 15 pg_rewind | Fichier : docs/schemas-patroni-odoo14/09-retour-ancien-primaire.drawio

Figure 10 — Transition : du cluster initial au mode Patroni

ACTUEL — configuration du 3 novembre 2026 ; primaire illustratif



Lecture : Cette planche raconte l'adoption de l'ancien cluster pg_server : préparation, certificats et etcd, détachement du service Debian, reprise des données par Patroni, rattachement des répliques puis configuration des clients.

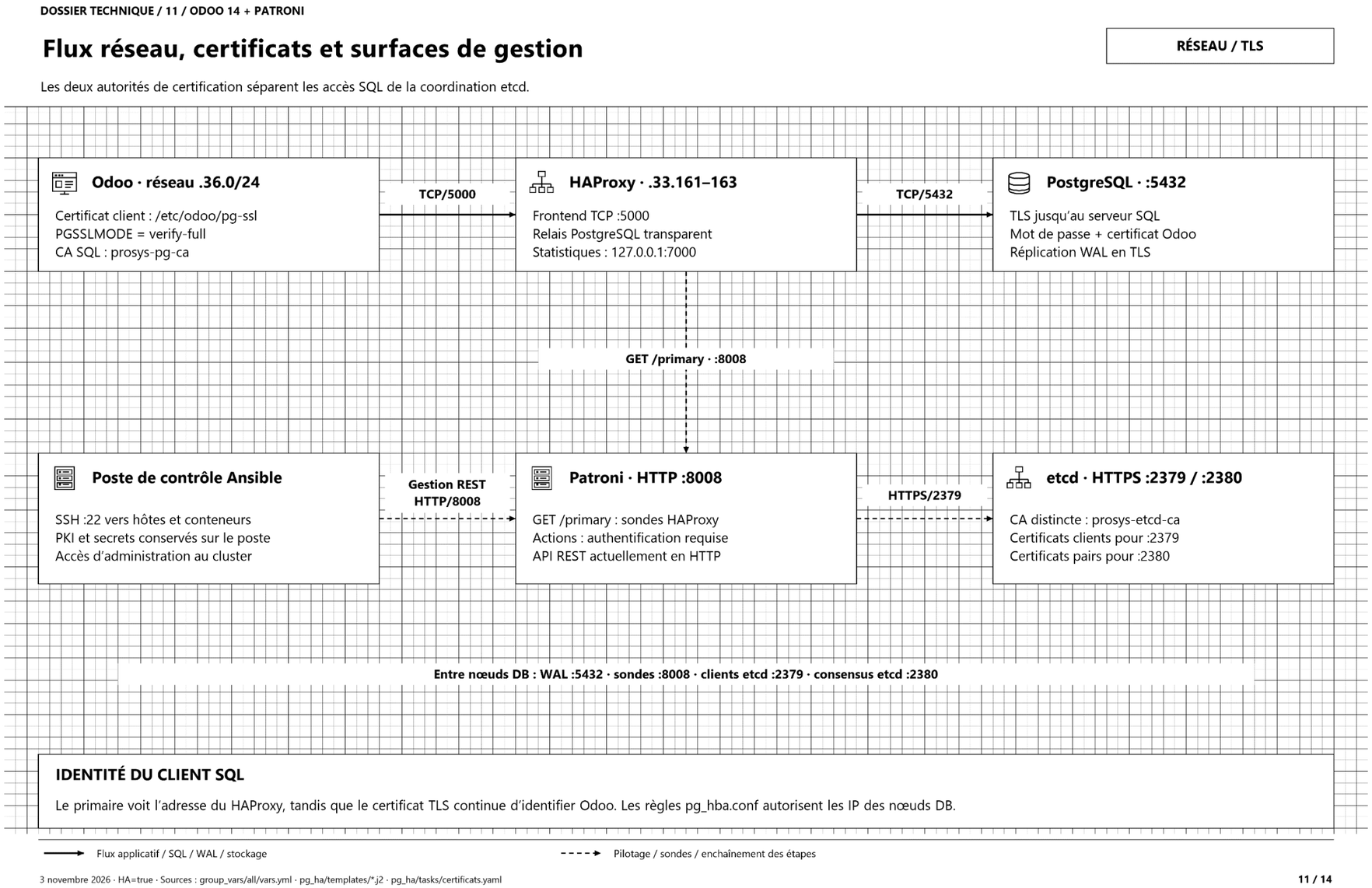
Fonctionnement : La transition cherche à conserver les données existantes et à vérifier chaque endpoint avant l'utilisation normale du chemin HA. Le playbook Odoo actuel sélectionne déjà pg_ha.

À retenir : La transition est un parcours historique ; l'activation n'est pas présentée comme restant à faire. Les trois membres doivent être considérés ensemble lors de la reprise initiale.

Source : playbooks/ansi_postgresql_ha.yaml · pg_ha/tasks/{preflight,reprise,configuration,verif}.yaml | Fichier : docs/schemas-patroni-odoo14/10-transition-vers-patroni.drawio

Figure 11 — Flux réseau, certificats et surfaces de gestion

ACTUEL — configuration du 3 novembre 2026 ; primaire illustratif



Lecture : La ligne supérieure représente le SQL chiffré de bout en bout. La ligne inférieure sépare la gestion REST Patroni de la coordination etcd. Les ports et les autorités de certificats sont indiqués sur chaque liaison.

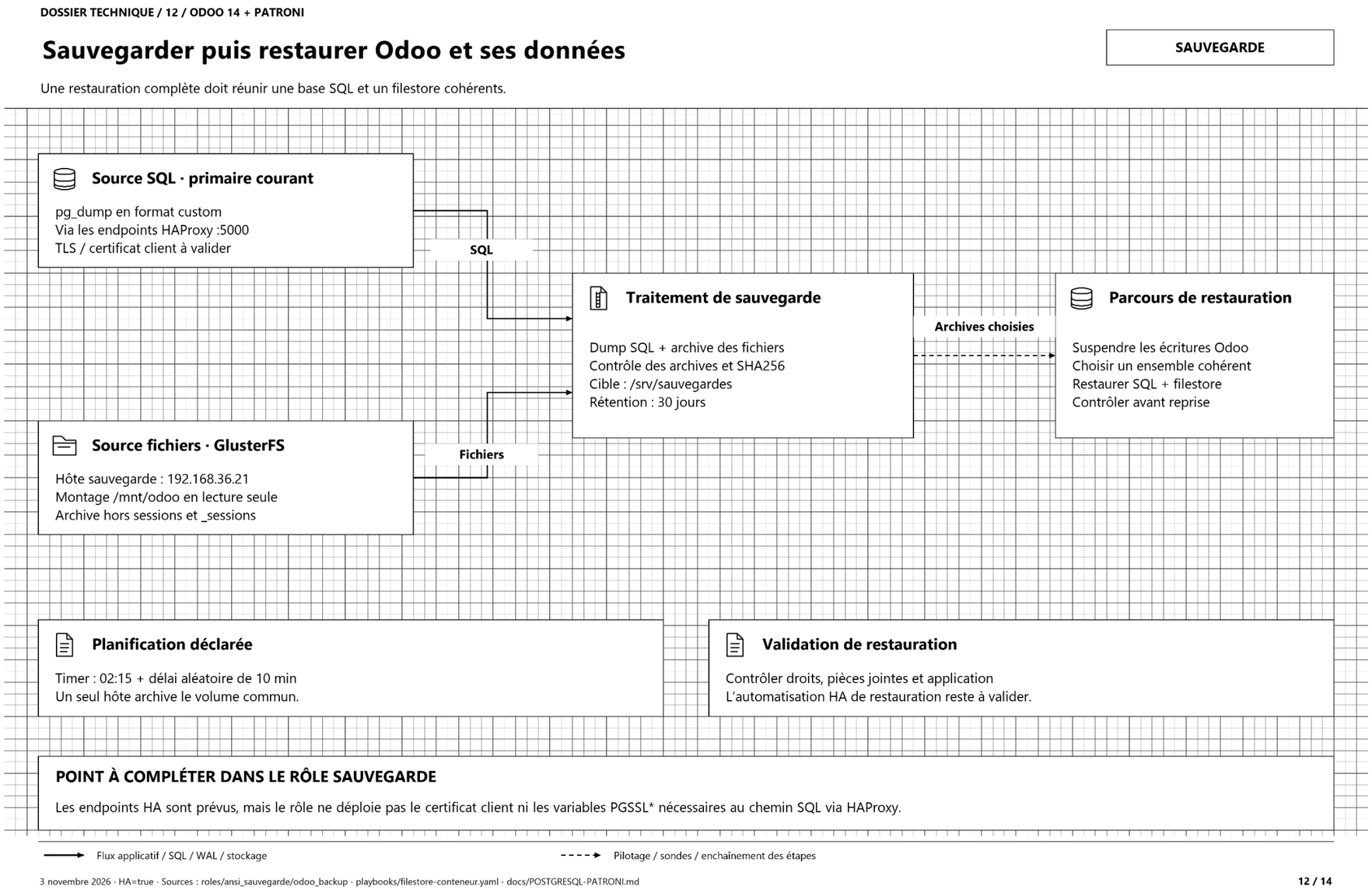
Fonctionnement : Le primaire voit la connexion réseau du proxy, mais le certificat client identifie Odoo. Etcd utilise sa propre autorité. Les statistiques HAProxy sont limitées à 127.0.0.1:7000.

À retenir : L'API Patroni demeure en HTTP/8008 avec authentification pour les actions. SSH/22 administre les systèmes ; il ne constitue pas le protocole de l'API Patroni.

Source : group_vars/all/vars.yml · pg_ha/templates/*.j2 · pg_ha/tasks/certificats.yaml | Fichier : docs/schemas-patroni-odoo14/11-flux-reseau-et-tls.drawio

Figure 12 — Sauvegarder puis restaurer Odoo et ses données

ACTUEL — configuration du 3 novembre 2026 ; primaire illustratif



Lecture : Les deux branches d'entrée sont complémentaires : le dump SQL lit le primaire courant et l'archive lit le volume GlusterFS. Elles conduisent à un ensemble de sauvegarde contrôlé avant une restauration choisie.

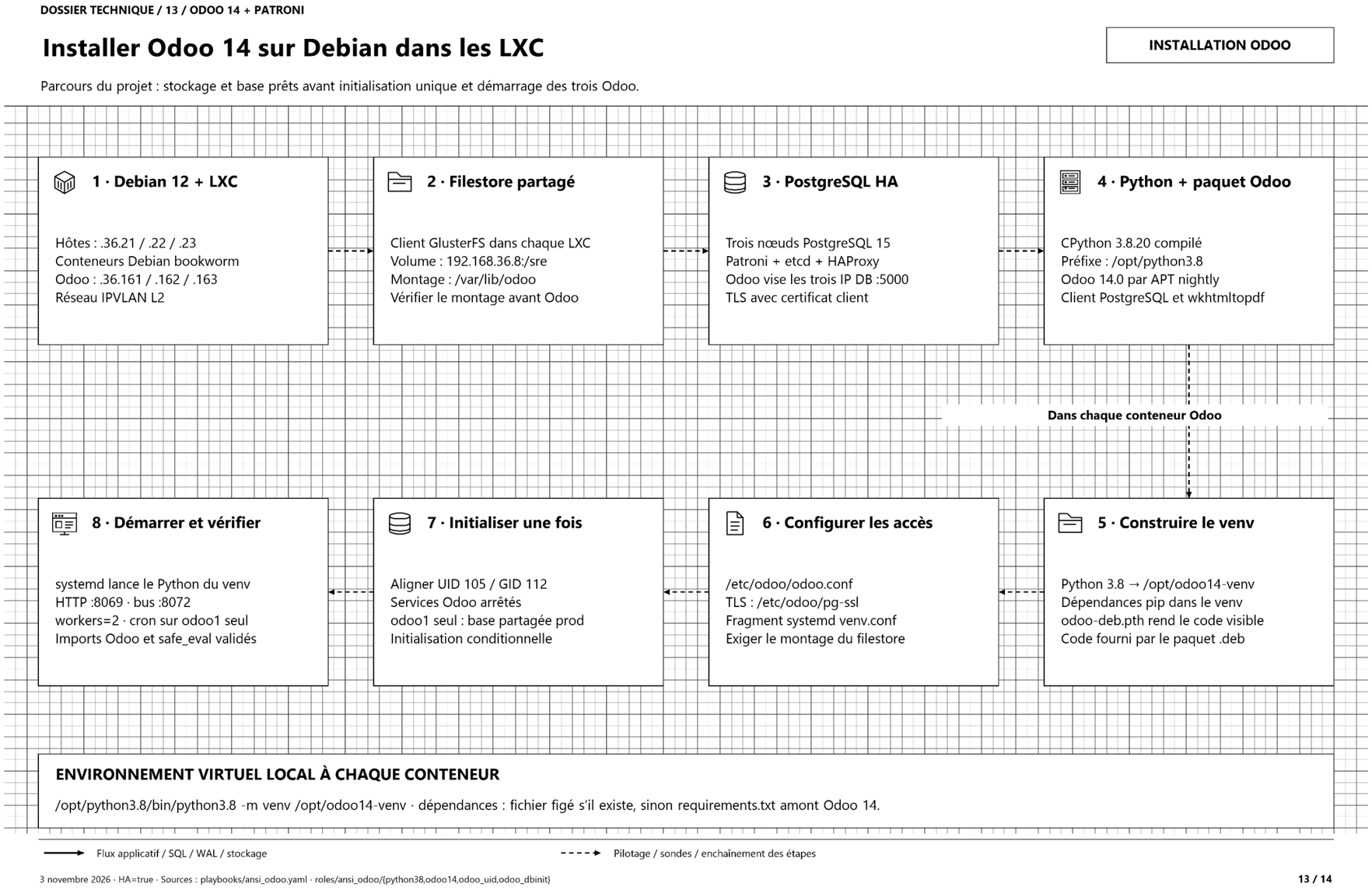
Fonctionnement : Le rôle prévoit /srv/sauvegardes, un manifeste SHA256 et une rotation à 30 jours. La restauration doit réunir des données SQL et fichiers cohérents, avec arrêt des écritures pendant l'opération.

À retenir : Le déploiement du certificat client SQL et des variables PGSSL* de sauvegarde reste incomplet. Le flux via HAProxy ne doit pas être considéré comme validé opérationnel.

Source : roles/ansi_sauvegarde/odoo_backup · playbooks/filestore-conteneur.yaml · docs/POSTGRESQL-PATRONI.md | Fichier : docs/schemas-patroni-odoo14/12-sauvegarde-et-restauration.drawio

Figure 13 — Installer Odoo 14 sur Debian dans les LXC

ACTUEL — configuration du 3 novembre 2026 ; primaire illustratif



Lecture : Suivre les huit étapes en allant vers la droite sur la première ligne, puis en revenant vers la gauche. Le réseau, le volume partagé et la base précèdent la configuration du venv et l'initialisation applicative.

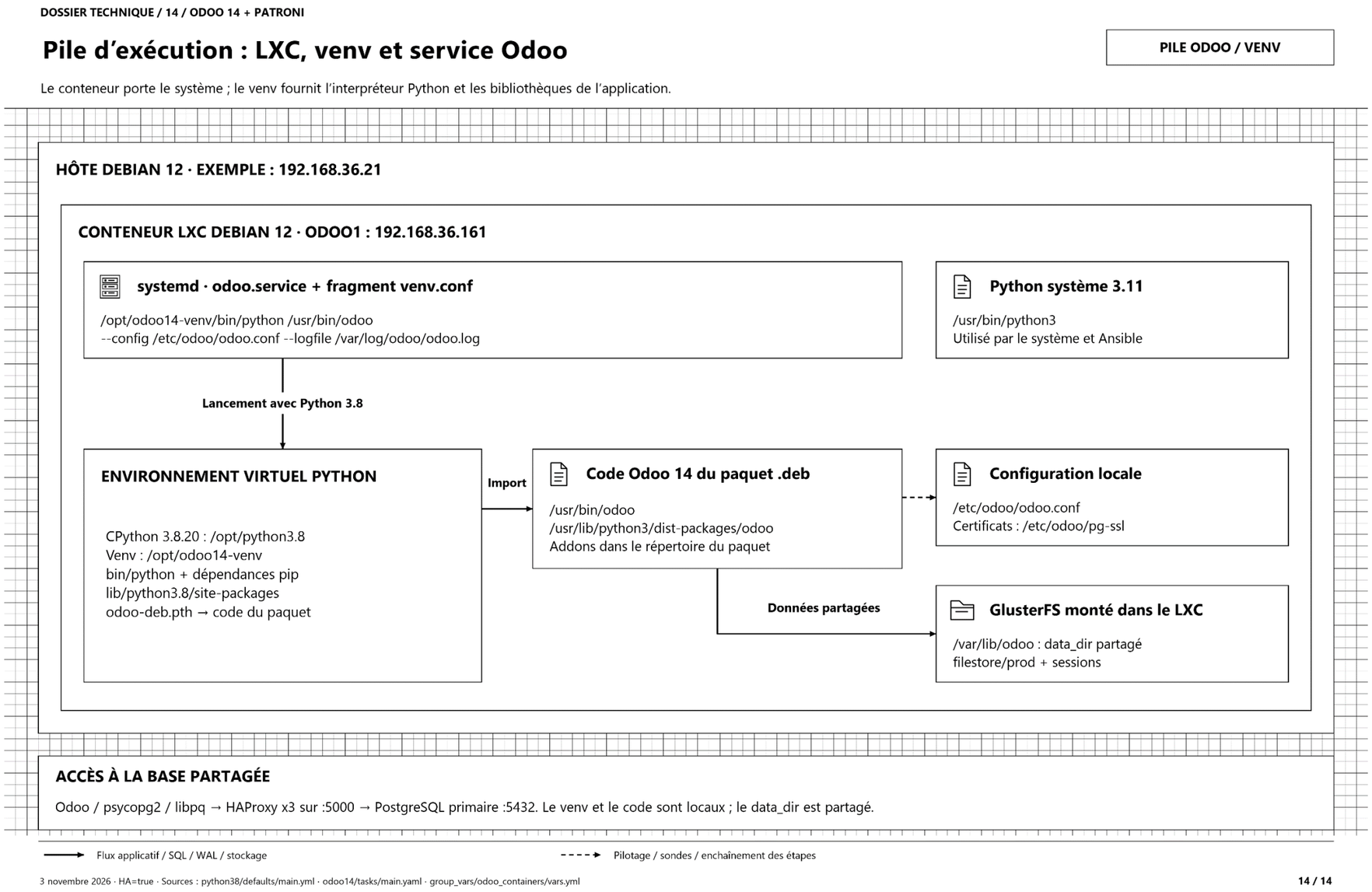
Fonctionnement : Le paquet Debian fournit le code Odoo ; CPython 3.8.20 et le venv fournissent l'exécution et les bibliothèques. UID 105 et GID 112 sont communs pour accéder aux fichiers partagés.

À retenir : L'initialisation est conditionnelle et ne doit être effectuée que par odoo1, services arrêtés, avant le démarrage des trois applications.

Source : playbooks/ansi_odoo.yaml · roles/ansi_odoo/{python38,odoo14,odoo_uid,odoo_dbinit} | Fichier : docs/schemas-patroni-odoo14/13-installation-odoo14-debian-lxc.drawio

Figure 14 — Pile d’exécution : LXC, venv et service Odoo

ACTUEL — configuration du 3 novembre 2026 ; primaire illustratif



Lecture : Les cadres sont emboîtés : hôte, conteneur, processus et environnement Python. systemd lance explicitement le Python du venv ; odoo-deb.pth rend le code du paquet Debian importable depuis celui-ci.

Fonctionnement : Le venv, le code, la configuration et les journaux sont locaux. Le data_dir /var/lib/odoo est partagé et contient notamment filestore/prod et sessions. Le Python système reste disponible pour Debian et Ansible.

À retenir : Un venv n’est pas un conteneur supplémentaire. La base SQL et le volume de fichiers sont deux dépendances distinctes de la même application.

Source : python38/defaults/main.yml · odoo14/tasks/main.yaml · group_vars/odoo_containers/vars.yml | Fichier : docs/schemas-patroni-odoo14/14-pile-execution-odoo14-venv.drawio

8. Atlas historique du projet

Les treize figures suivantes existaient avant le dernier dossier de quatorze vues. Elles sont reproduites pour conserver toute la documentation graphique du projet. Le cartouche de chaque page indique ce qui reste utile et ce qui a changé.

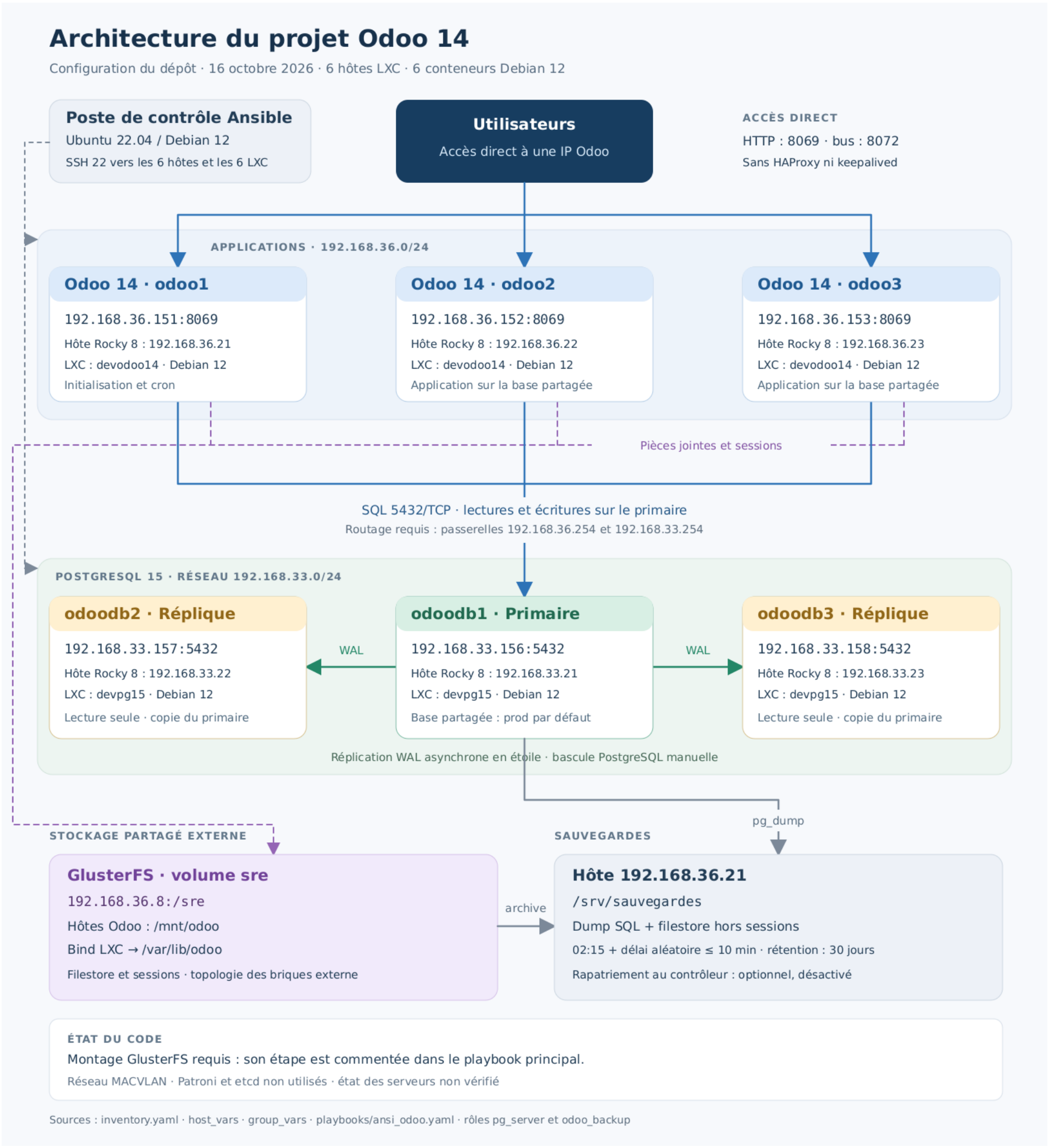
Les différences majeures sont le passage au mode Patroni actif, le chemin SQL via HAProxy sur 5000, la sélection IPVLAN et le montage direct du volume dans les LXC Odoo. Les anciennes mentions de bases locales, de Rocky, de montage hôte/bind ou de bascule manuelle ne décrivent pas toutes la configuration actuelle.

Les fichiers draw.io à plusieurs onglets et leurs PNG/SVG sont plusieurs formats des mêmes vues. Ils ne sont pas comptés plusieurs fois ; le classeur expert répète lui aussi une vue déjà incluse.

Époque	Évolution représentée
16 octobre 2026	Première synthèse : base partagée, deux répliques et bascule manuelle.
18 octobre 2026	Vues LXC, volume GlusterFS, bind hôte et réseau d’ensemble.
25 octobre 2026	Comparaison de l’existant avec l’option Patroni ; HA encore indiquée désactivée dans ces dessins.
3 novembre 2026	HA sélectionnée, montage Gluster direct et venv explicités dans le nouveau dossier.

Figure 15 — Synthèse d’architecture du 16 octobre

HISTORIQUE — 16 octobre 2026



Lecture : Cette synthèse montre les trois applications, la base partagée, deux répliques, les fichiers communs et l’hôte de sauvegarde. Elle établit les grandes catégories de données et de flux.

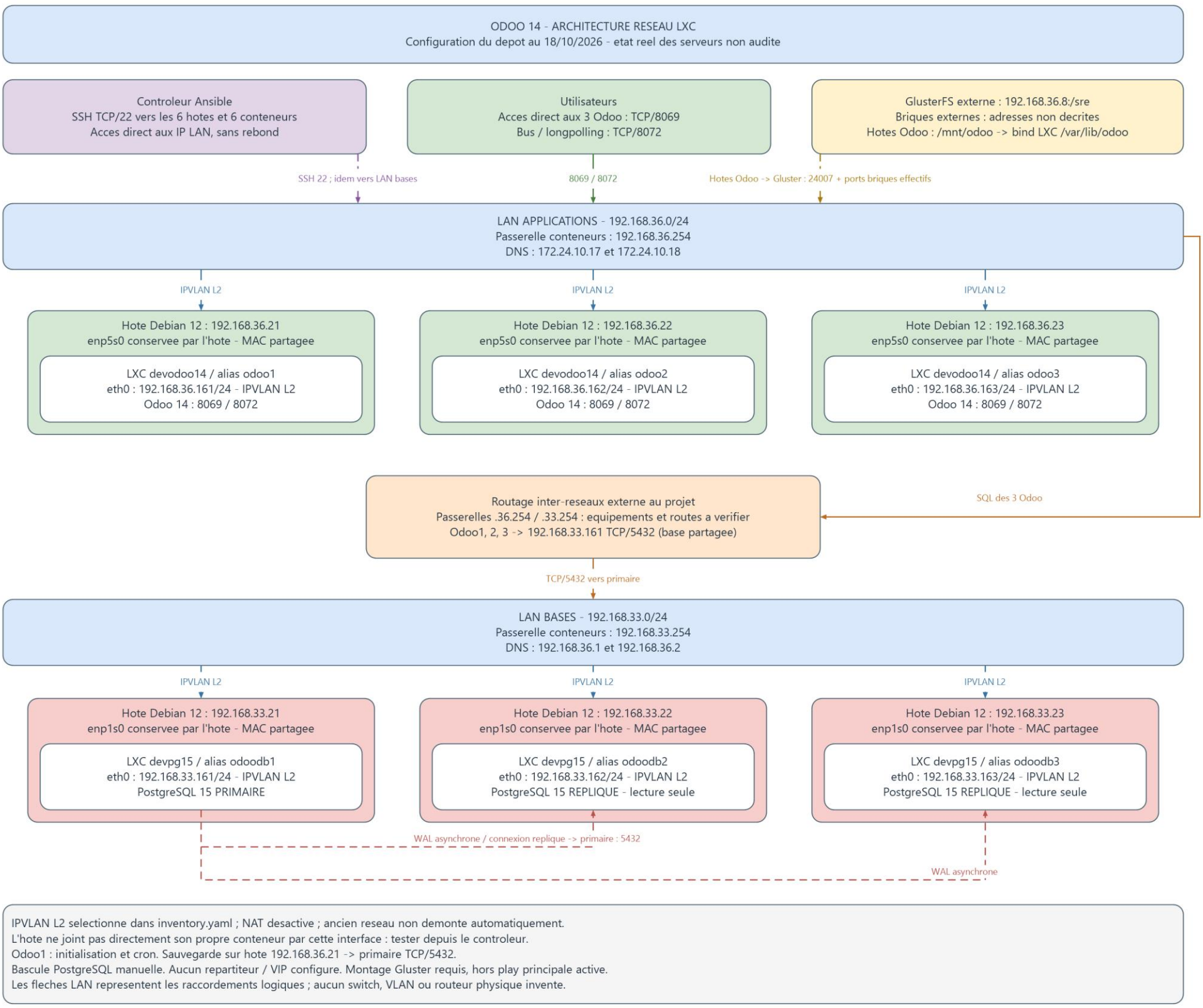
Fonctionnement : Le partage SQL et fichiers reste le principe actuel. Les mentions de Rocky, la connexion directe à .33.161:5432 et la bascule manuelle appartiennent à cette première version documentaire.

À retenir : Utiliser les figures 1 à 14 pour les endpoints HA et le montage actuels. Cette ancienne figure ne doit pas être lue comme une seconde configuration simultanément active.

Source : docs/ARCHITECTURE-ACTUELLE.md | Fichier : docs/architecture-actuelle.mmd

Figure 16 — Architecture réseau LXC d’ensemble

HISTORIQUE — 18 octobre 2026 ; export du draw.io original



Lecture : Les deux bandes LAN relient les six hôtes et leurs conteneurs. Les liaisons supérieures distinguent utilisateurs, administration et stockage. Le cadre central représente le routage externe.

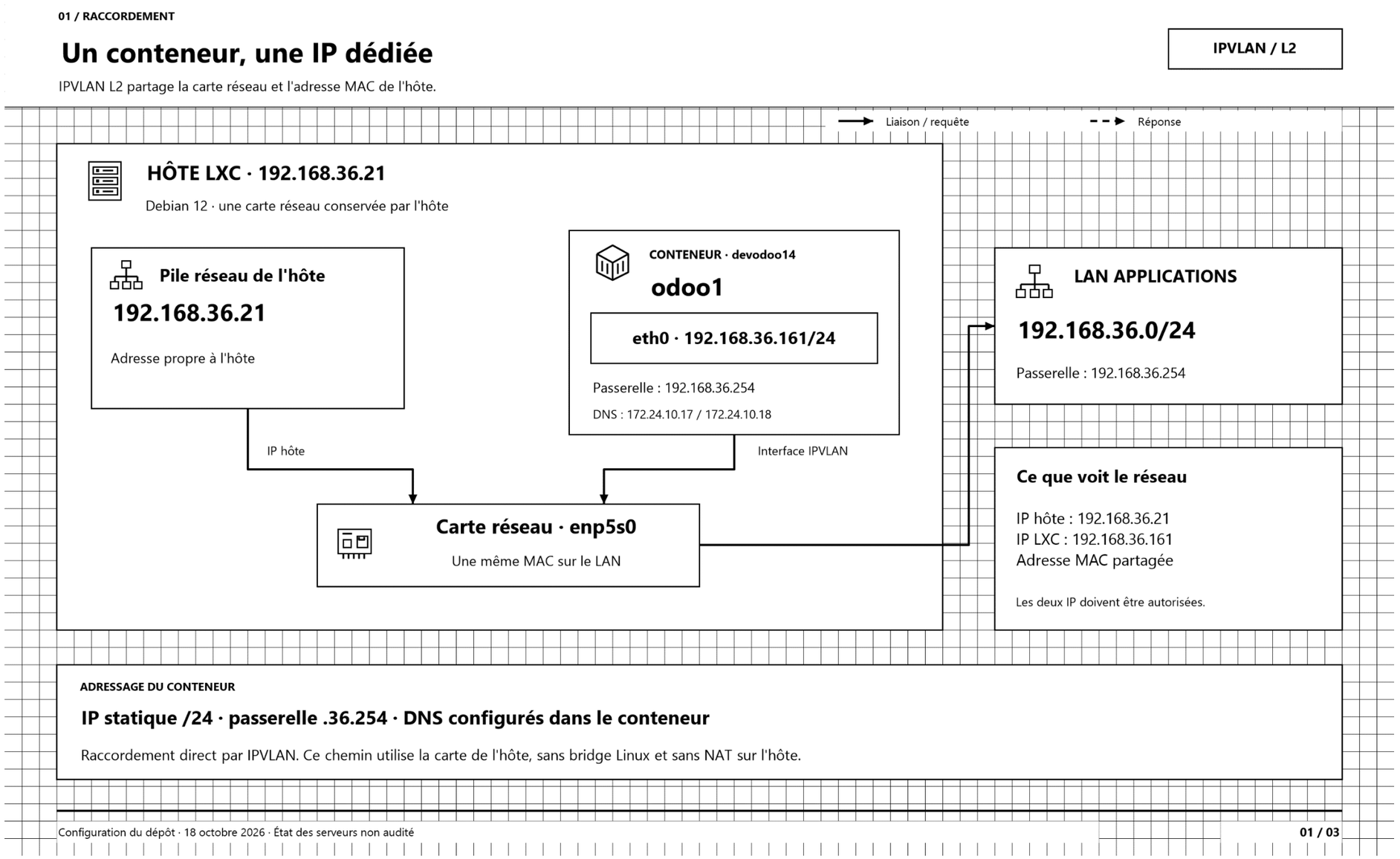
Fonctionnement : Les adresses et le principe IPVLAN restent utiles. Le SQL direct vers un primaire fixe, les montages Gluster hôtes et les notes de bascule manuelle décrivent l’état de création du document.

À retenir : Le rapport fournit ici le rendu d’une vue qui n’avait pas de PNG dédié. Le source draw.io est conservé et n’a pas été modifié.

Source : docs/architecture-reseau-lxc.drawio | Fichier : docs/architecture-reseau-lxc.drawio

Figure 17 — Raccordement IPVLAN L2

RÉSEAU — vue du 18 octobre 2026, principe toujours pertinent



Lecture : Le conteneur possède son interface eth0 et une IP propre. IPVLAN la raccorde à la carte parente enp5s0, que l’hôte continue d’utiliser. La MAC est partagée ; les adresses IP restent distinctes.

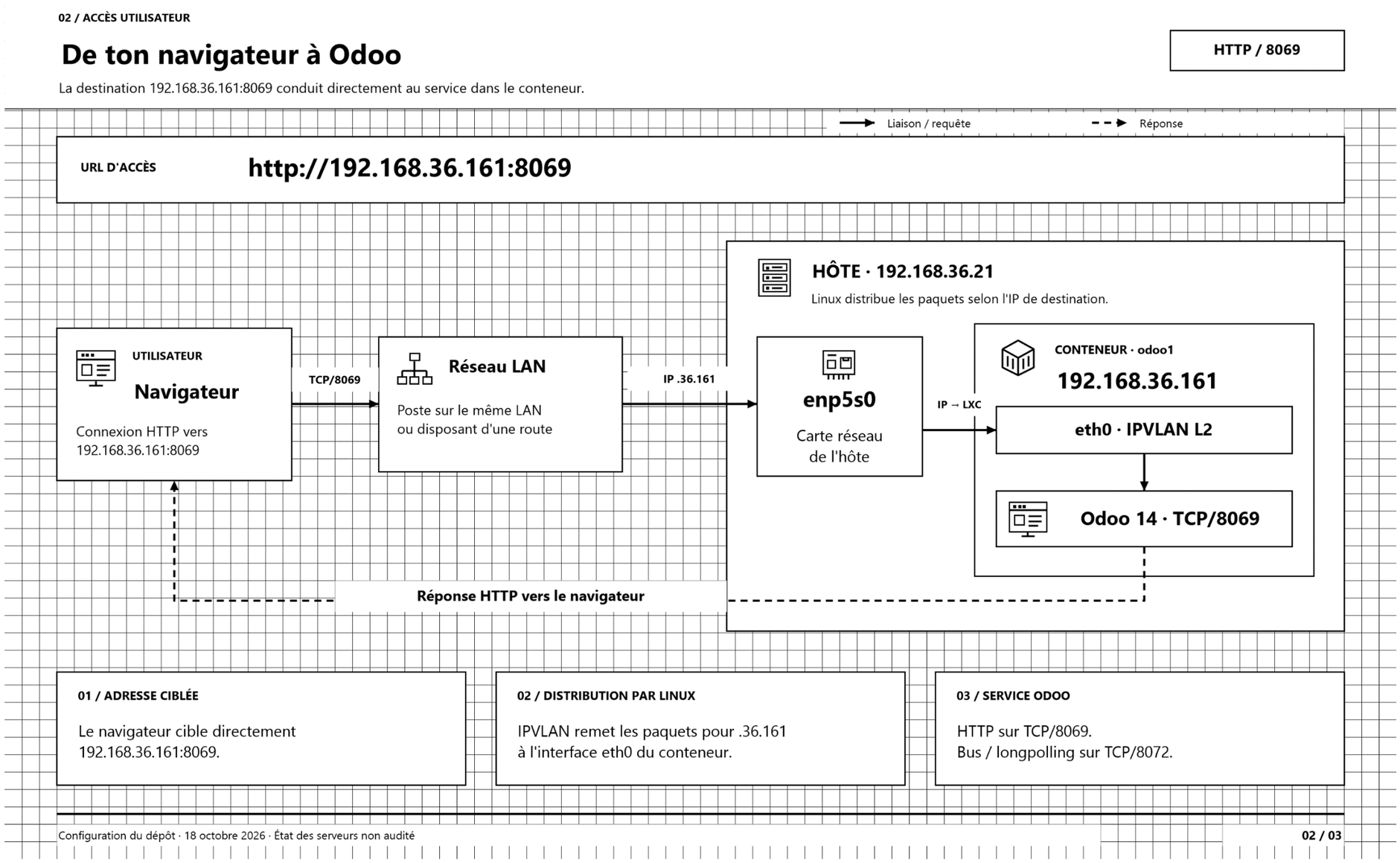
Fonctionnement : Cette logique correspond encore aux valeurs exécutables lxc_net_type=ipvlan et lxc_egress_nat=false. Le chemin n’exige pas un bridge L2 ni un NAT de l’hôte.

À retenir : L’hôte ne joint pas directement son propre LXC par l’interface IPVLAN parente. Un contrôle doit utiliser un chemin adapté, par exemple un autre poste routé ou lxc-attach.

Source : docs/schemas-lxc/README.md ; inventory.yaml | Fichier : docs/schemas-lxc/architecture-lxc-professionnelle.drawio

Figure 18 — Accès utilisateur à Odoo

RÉSEAU — vue du 18 octobre 2026, accès direct toujours pertinent



Lecture : Le navigateur vise l'IP du conteneur sur 8069. Le réseau remet le trafic à l'interface IPVLAN du LXC ; l'IP de l'hôte ne constitue pas une redirection web dans ce chemin.

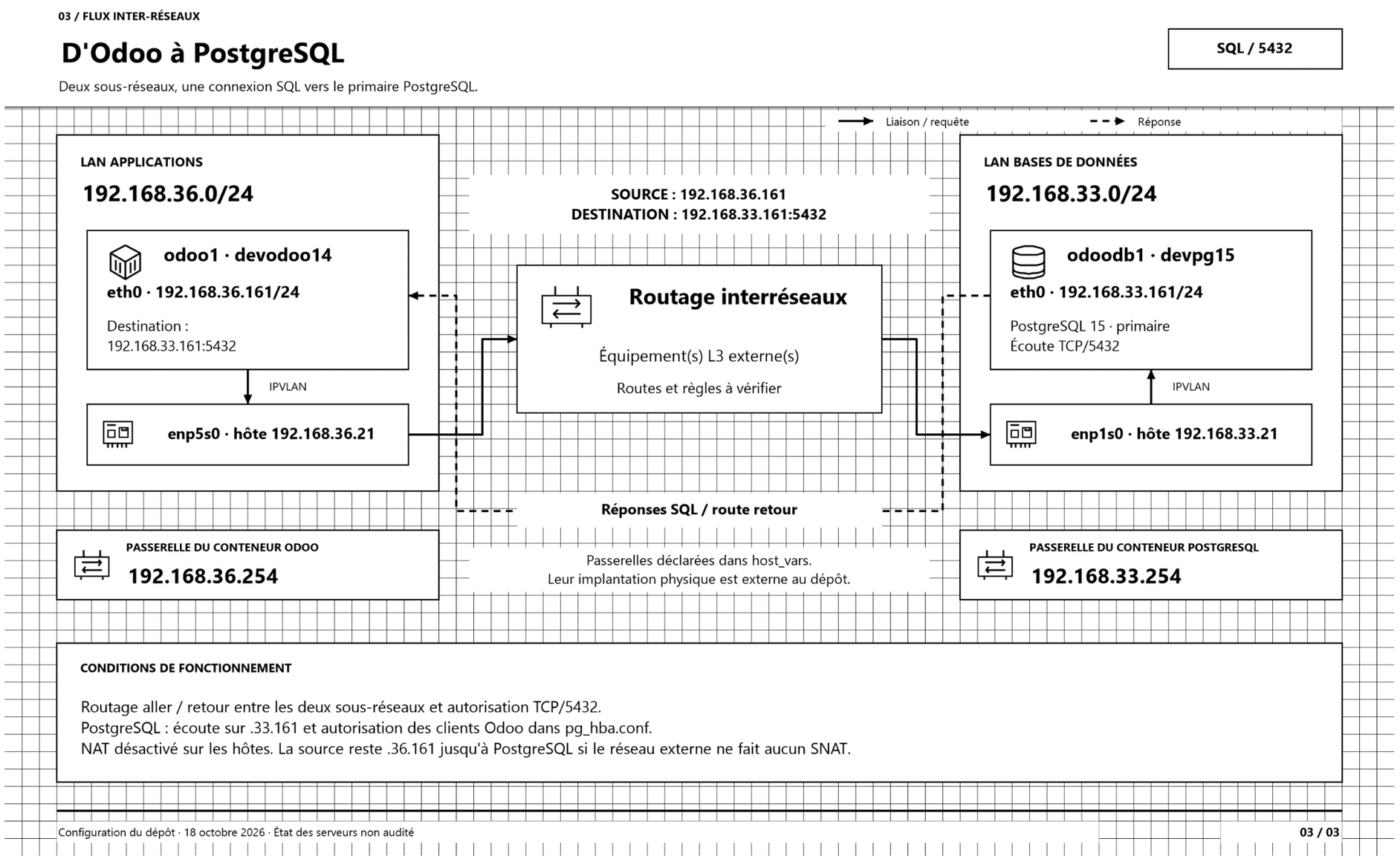
Fonctionnement : Le port 8072 porte le bus de notifications avec les workers. Le parcours actuel conserve l'accès direct aux applications ; les HAProxy PostgreSQL ne servent pas de répartiteurs HTTP.

À retenir : Plusieurs applications disponibles ne créent pas automatiquement une adresse web unique. L'exemple Nginx du dépôt reste une proposition à distinguer de ce chemin.

Source : docs/schemas-lxc/README.md ; group_vars/all/vars.yml | Fichier : docs/schemas-lxc/architecture-lxc-professionnelle.drawio

Figure 19 — Ancien flux Odoo vers PostgreSQL

HISTORIQUE — destination SQL antérieure à la HA



Lecture : Le schéma explique le passage du réseau applicatif .36.0/24 au réseau des bases .33.0/24. Dans sa version d’origine, l’application vise .33.161:5432.

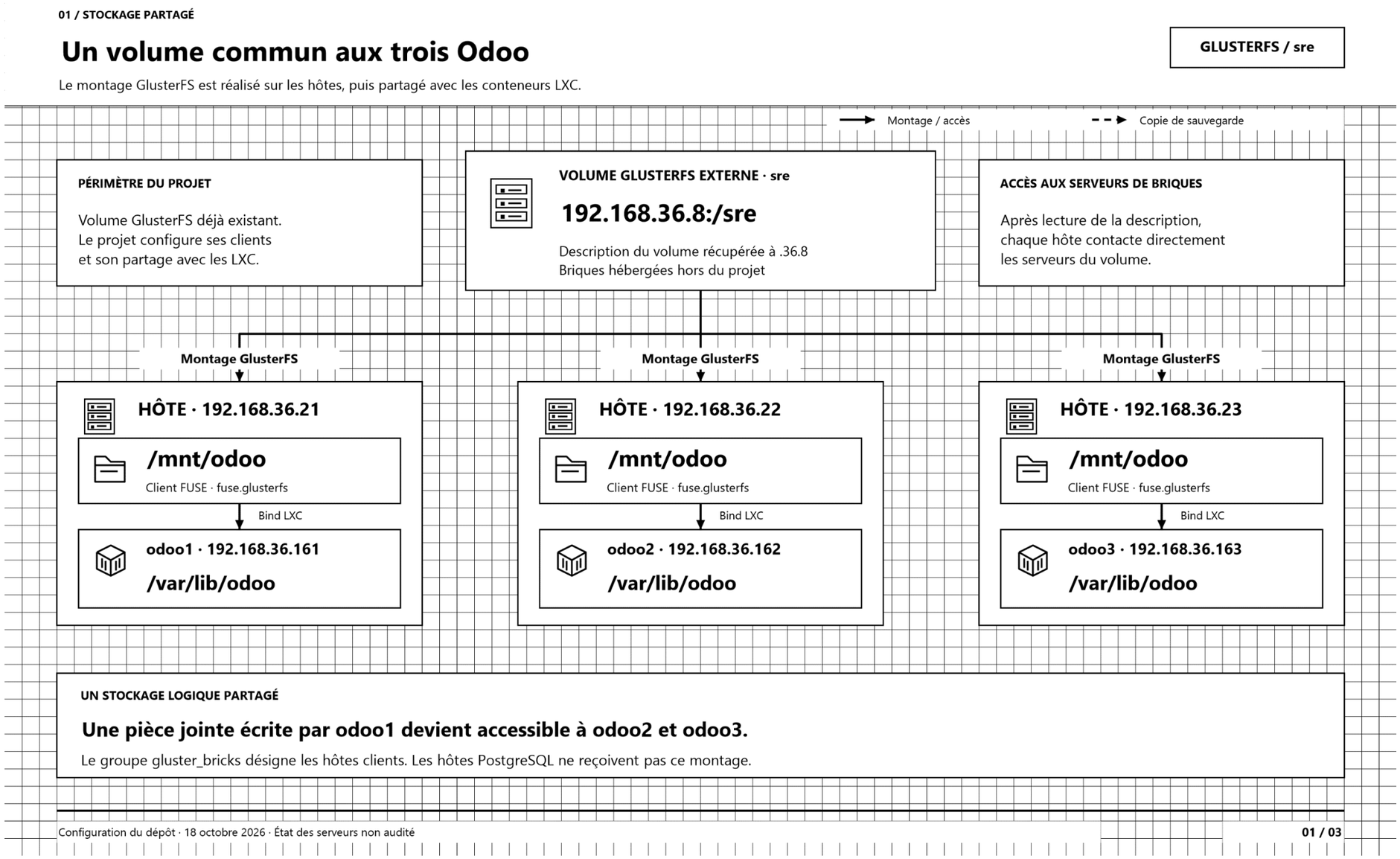
Fonctionnement : Le routage externe reste nécessaire, mais la destination cliente est aujourd’hui la liste des trois HAProxy sur 5000. Ceux-ci choisissent le primaire courant au lieu d’imposer un primaire fixe.

À retenir : L’absence de NAT sur les hôtes n’exclut pas le SNAT du routeur externe. Le certificat client conserve l’identité SQL d’Odoo à travers le chemin actuel.

Source : docs/schemas-lxc/README.md ; group_vars/all/vars.yml | Fichier : docs/schemas-lxc/architecture-lxc-professionnelle.drawio

Figure 20 — Volume GlusterFS partagé par trois Odoo

HISTORIQUE — principe partagé actuel, chaîne de montage ancienne



Lecture : Les trois instances retrouvent les mêmes pièces jointes et sessions dans le volume logique sre. Le serveur .36.8 fournit sa description ; les clients communiquent ensuite avec les serveurs de briques.

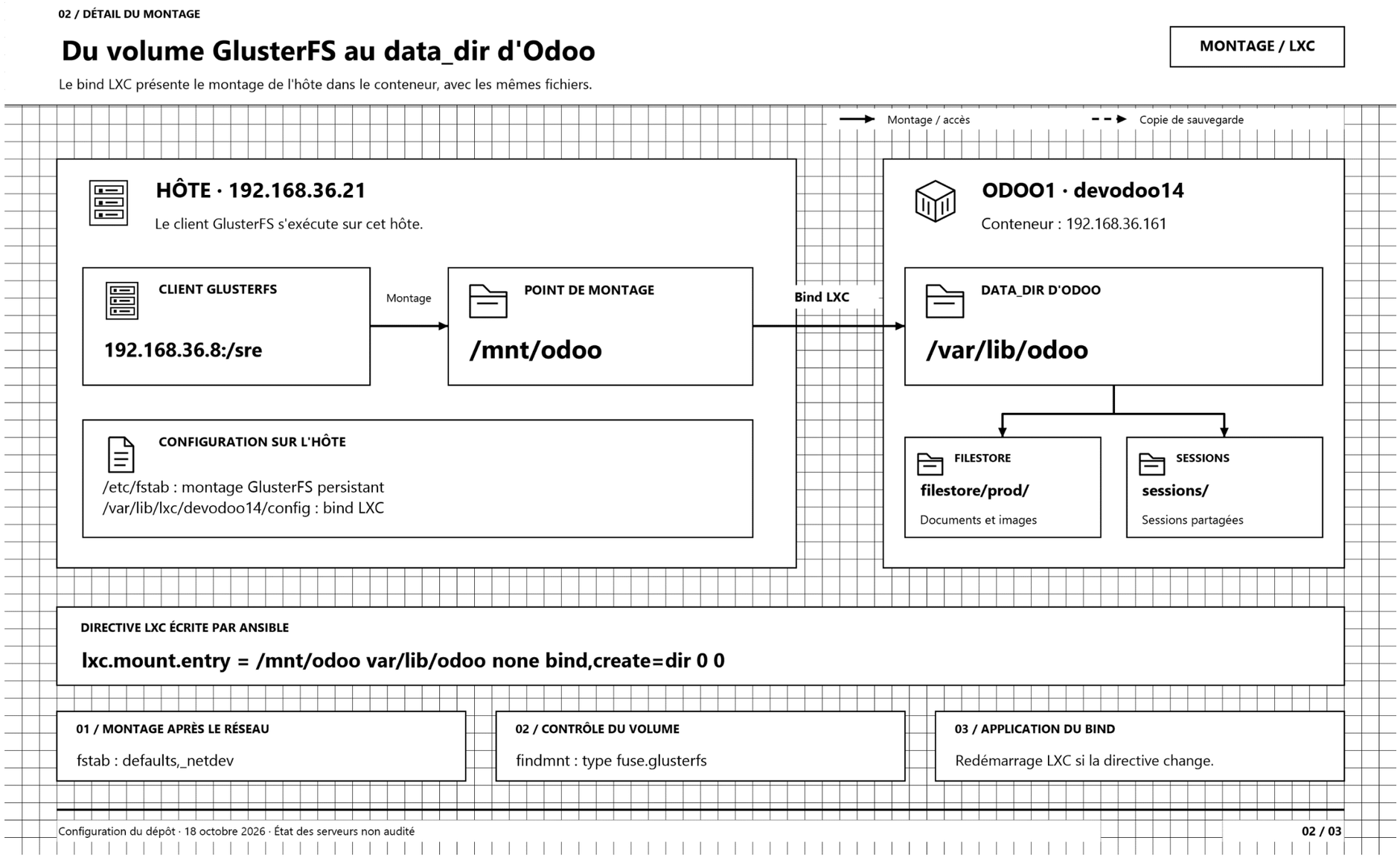
Fonctionnement : Le partage du data_dir reste actuel, mais les clients GlusterFS sont désormais dans les conteneurs Odoo. Les trois montages hôtes suivis de binds illustrent l'ancien mode d'accès.

À retenir : Le dépôt ne décrit pas la topologie complète des briques externes et ne permet pas d'en certifier les domaines de panne ou la disponibilité.

Source : docs/schemas-glusterfs/README.md ; playbooks/filestore-conteneur.yaml | Fichier : docs/schemas-glusterfs/architecture-glusterfs-professionnelle.drawio

Figure 21 — Ancien montage hôte et bind LXC

HISTORIQUE — mode remplacé par le montage direct dans les LXC



Lecture : Suivre la chaîne initiale : volume externe, montage /mnt/odoo sur l'hôte, puis bind vers /var/lib/odoo dans le conteneur. Le bind expose les mêmes fichiers ; ce n'est pas une seconde copie.

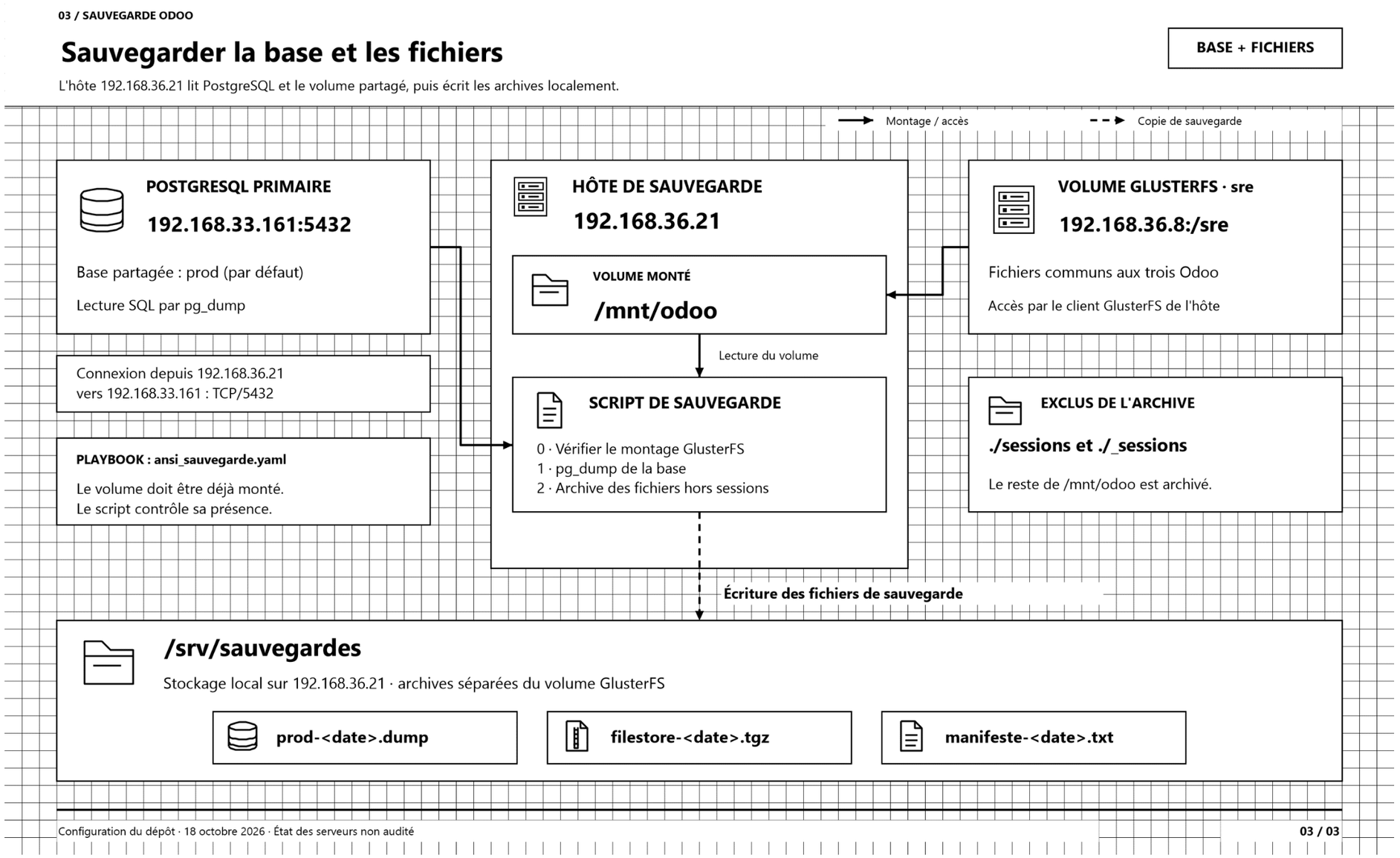
Fonctionnement : Le playbook actif retire cet ancien bind, rend /dev/fuse disponible dans le conteneur et inscrit le montage GlusterFS dans son /etc/fstab.

À retenir : Le garde-fou doit porter sur le véritable montage utilisé par Odoo. Un simple répertoire présent n'est pas la preuve que le volume partagé est disponible.

Source : docs/schemas-glusterfs/README.md ; roles/filestore-conteneur/filestore_conteneur | Fichier : docs/schemas-glusterfs/architecture-glusterfs-professionnelle.drawio

Figure 22 — GlusterFS et sauvegarde

HISTORIQUE — organisation générale conservée, entrée SQL modifiée



Lecture : L’hôte de sauvegarde réunit un dump SQL, une archive des fichiers et un manifeste dans /srv/sauvegardes. Les sessions sont exclues de l’archive.

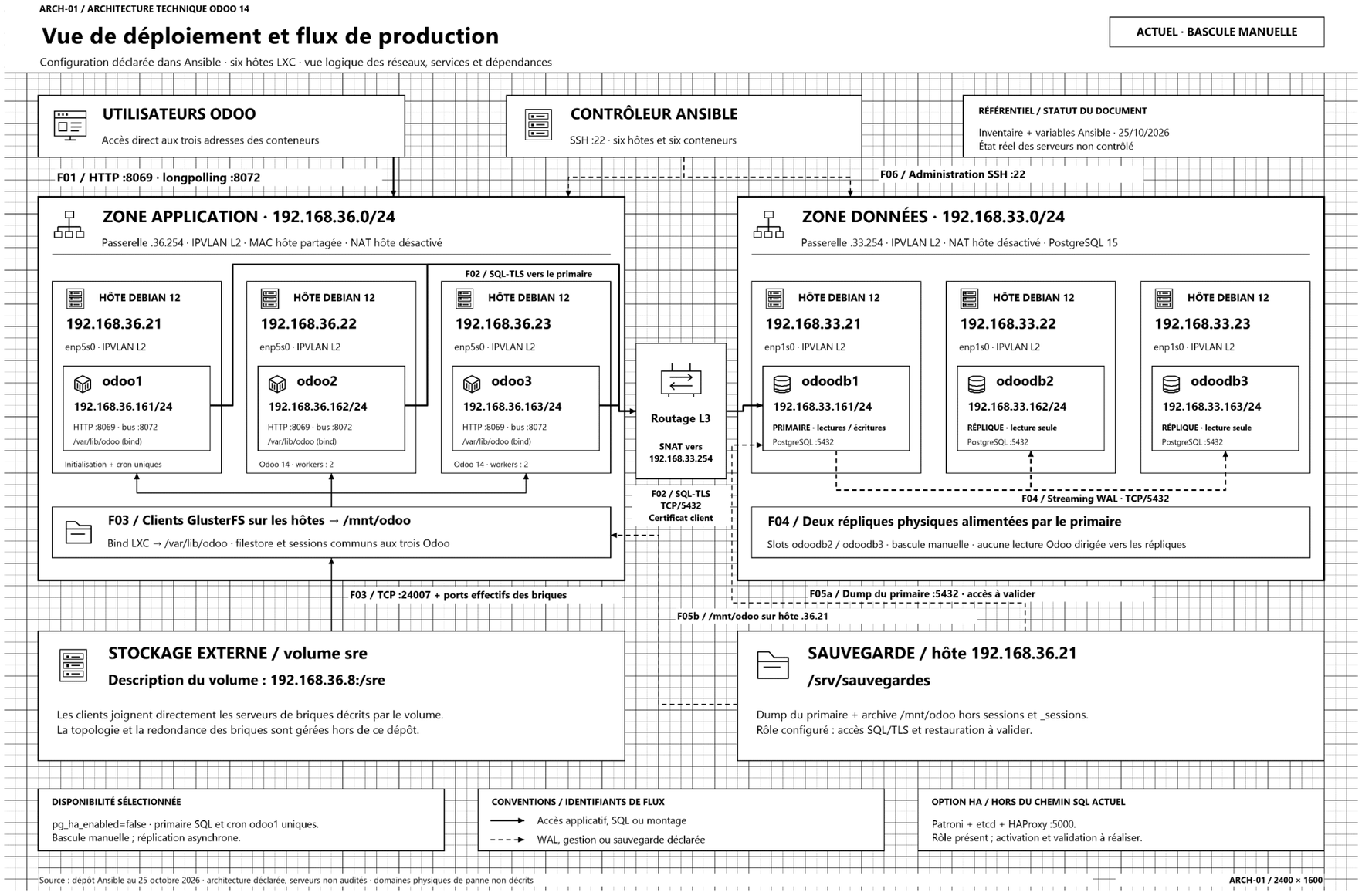
Fonctionnement : Cet hôte, .36.21, conserve aujourd’hui /mnt/odoo en lecture seule. Les deux autres hôtes Odoo cessent de monter le volume ; les applications le montent directement dans leurs LXC.

À retenir : La destination SQL fixe dessinée est historique. Le chemin HA est préparé, mais les certificats et variables TLS client du rôle de sauvegarde restent à compléter.

Source : docs/schemas-glusterfs/README.md ; roles/ansi_sauvegarde/odoo_backup | Fichier : docs/schemas-glusterfs/architecture-glusterfs-professionnelle.drawio

Figure 23 — Architecture technique avant activation HA

HISTORIQUE — configuration documentaire du 25 octobre 2026



Lecture : La figure détaille les frontières des hôtes et conteneurs, les zones réseau, les accès utilisateurs, les flux SQL, le stockage et la sauvegarde. Les identifiants de flux servent à suivre chaque dépendance.

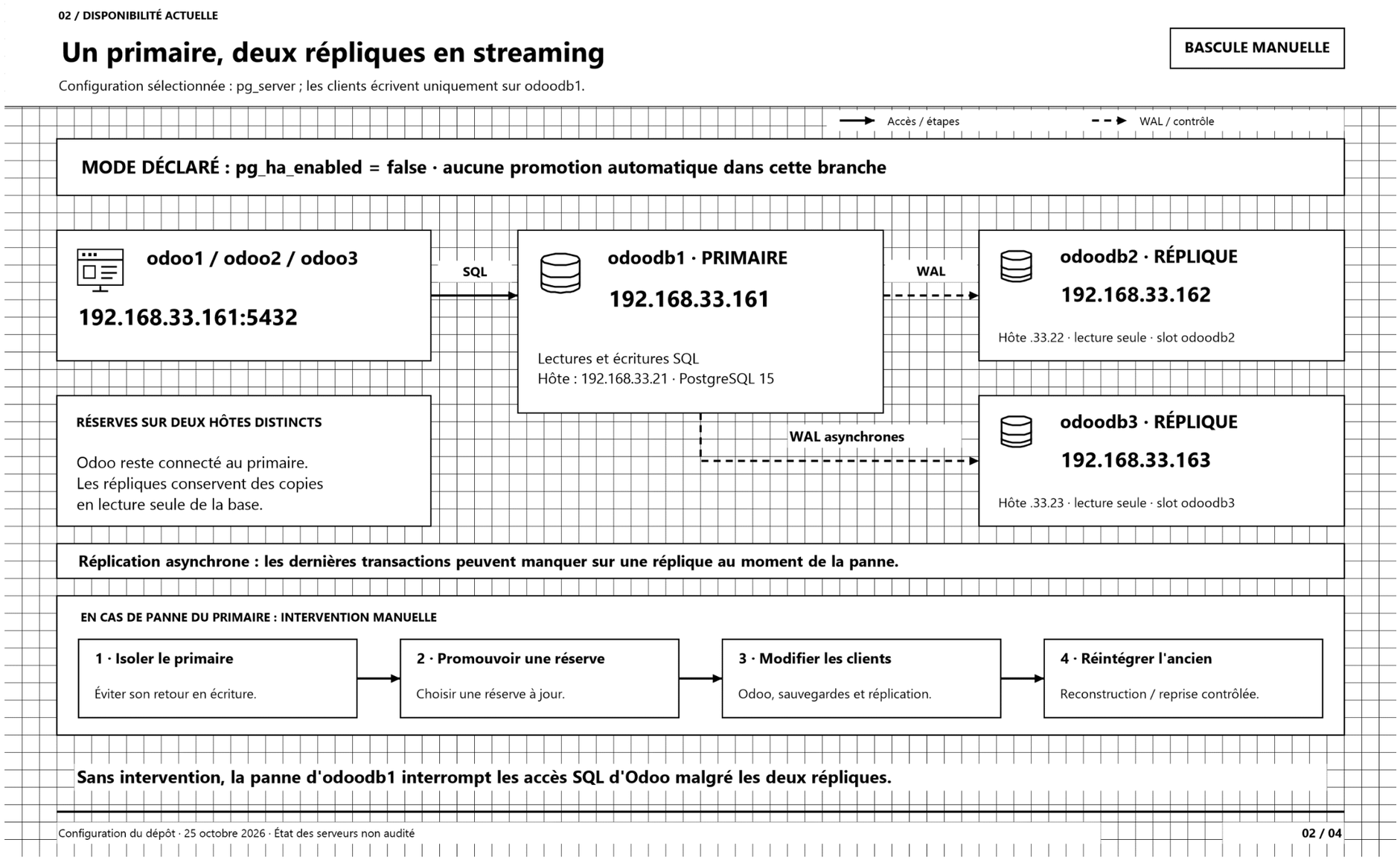
Fonctionnement : Elle décrivait pg_ha_enabled=false, le primaire fixe et les montages hôtes. La configuration actuelle sélectionne Patroni, utilise 5000 côté clients et monte GlusterFS dans chaque LXC Odoo.

À retenir : Le fichier architecture-actuelle-expert.drawio et la page 1 du classeur architecture HA reproduisent cette même vue ; ils ne sont pas deux figures supplémentaires.

Source : docs/schemas-architecture-ha/README.md | Fichier : docs/schemas-architecture-ha/architecture-actuelle-expert.drawio

Figure 24 — Réplication et bascule manuelle

HISTORIQUE — exploitation du cluster avant Patroni



Lecture : Un primaire fixé dans l’inventaire alimente deux répliques. En panne, la promotion et l’adaptation des connexions nécessitaient une intervention d’administration.

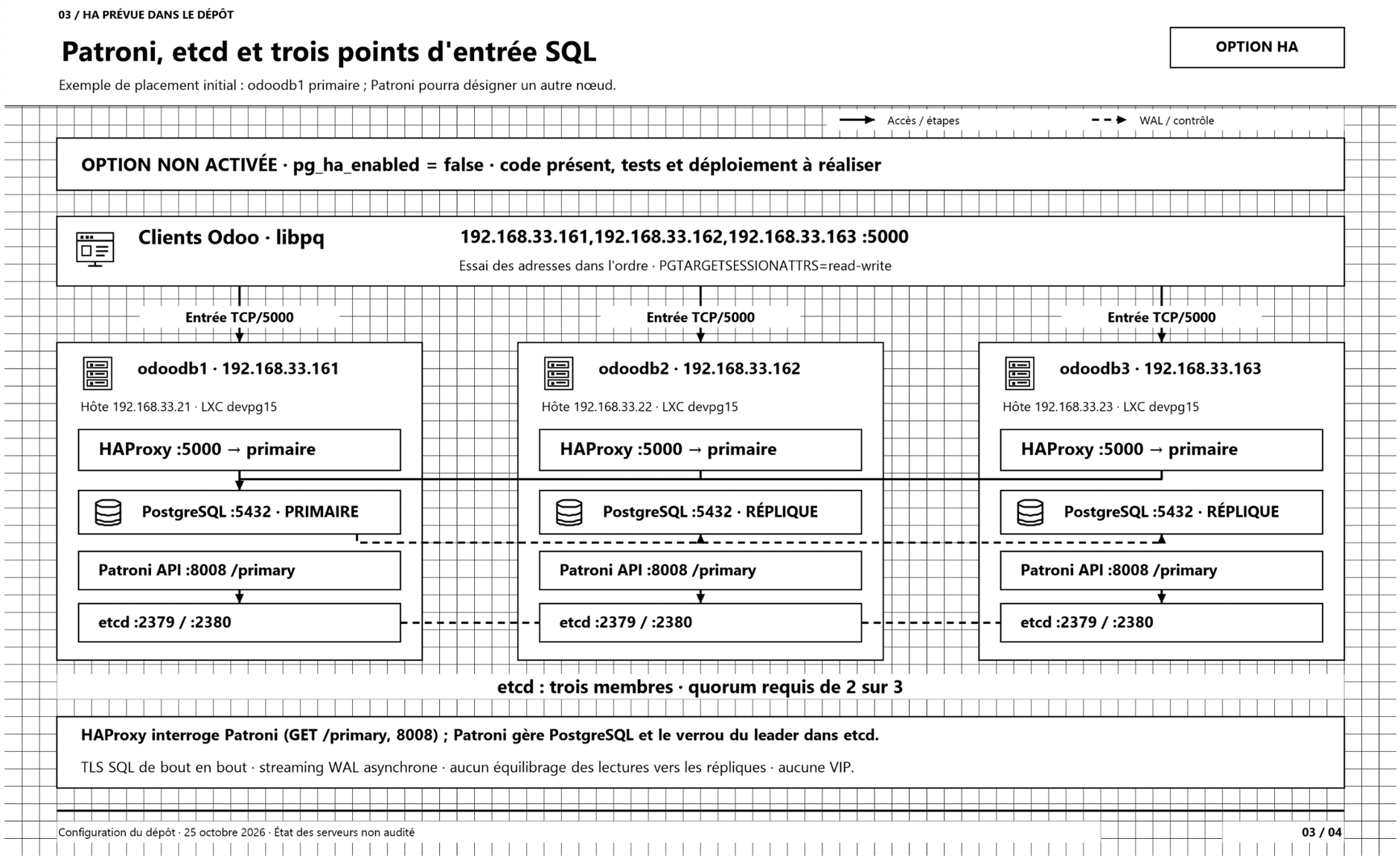
Fonctionnement : Cette vue explique pourquoi la réplication physique seule ne fournissait pas une reprise applicative automatique. Le routage client et la coordination de rôle ont ensuite été ajoutés par Patroni et HAProxy.

À retenir : Ne pas appliquer ce parcours manuel comme procédure nominale du cluster désormais géré par Patroni. L’ancien primaire doit toujours être maîtrisé avant tout retour.

Source : docs/schemas-architecture-ha/README.md | Fichier : docs/schemas-architecture-ha/architecture-actuelle-et-ha-postgresql.drawio

Figure 25 — Option HA Patroni, etcd et HAProxy

HISTORIQUE — HA présentée comme option au 25 octobre



Lecture : La vue distingue les applications, les proxies SQL, les agents Patroni et la coordination etcd. Elle introduit le mécanisme qui choisit dynamiquement le primaire.

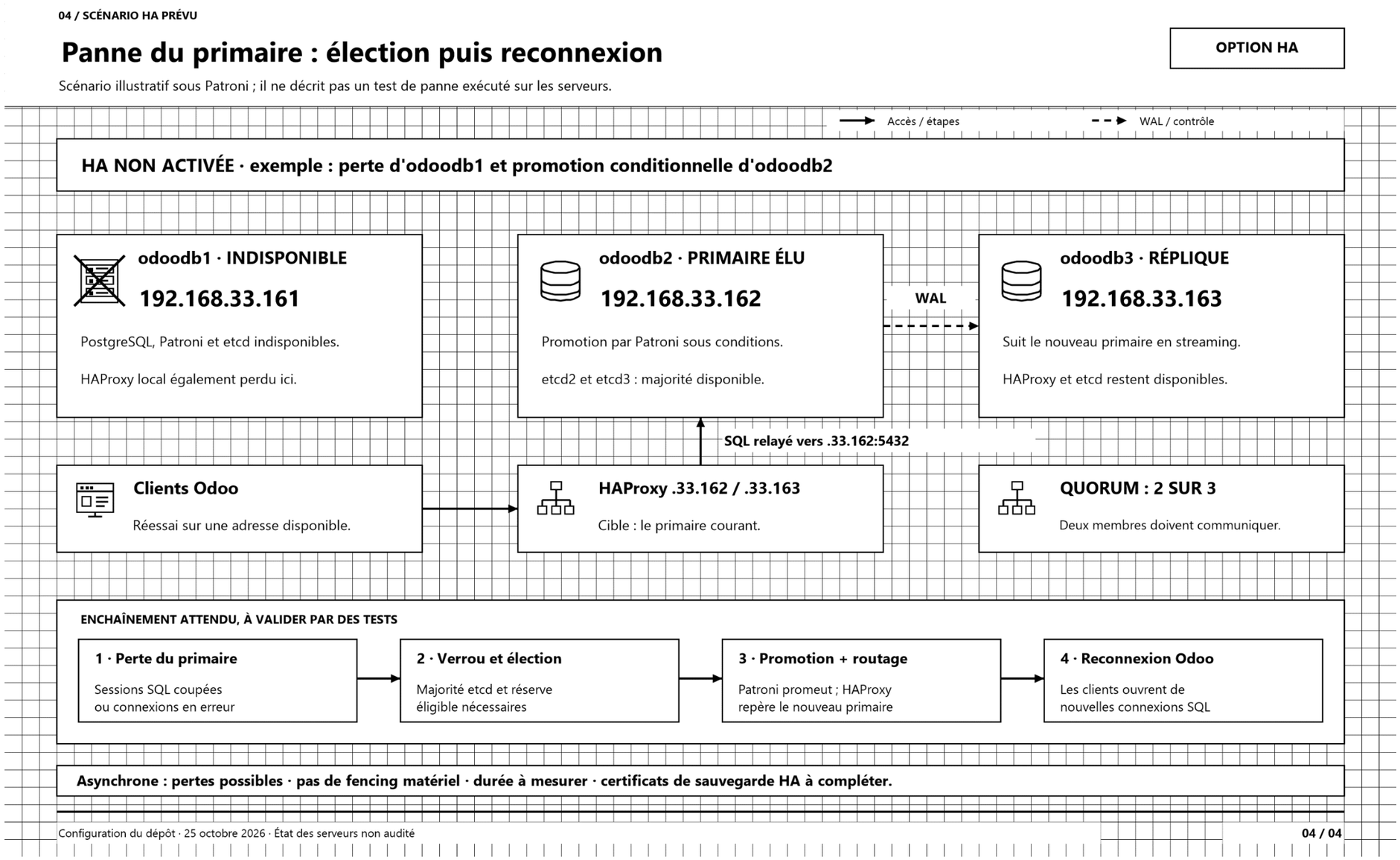
Fonctionnement : La topologie est celle retenue par le mode actuel, mais son cartouche d'origine décrivait une option désactivée. La valeur du dépôt est maintenant pg_ha_enabled=true.

À retenir : Une intention d'architecture et un résultat de test sont différents. Les tests de reprise et l'observation du primaire réel ne sont pas établis par le dessin.

Source : docs/schemas-architecture-ha/README.md ; group_vars/all/vars.yml | Fichier : docs/schemas-architecture-ha/architecture-actuelle-et-ha-postgresql.drawio

Figure 26 — Bascule HA et quorum — première vue

SCÉNARIO HISTORIQUE — mécanisme encore pertinent



Lecture : Le dessin illustre la perte d’un primaire et la décision d’un côté disposant encore de la majorité etcd. Un candidat éligible devient primaire, puis le routage SQL le suit.

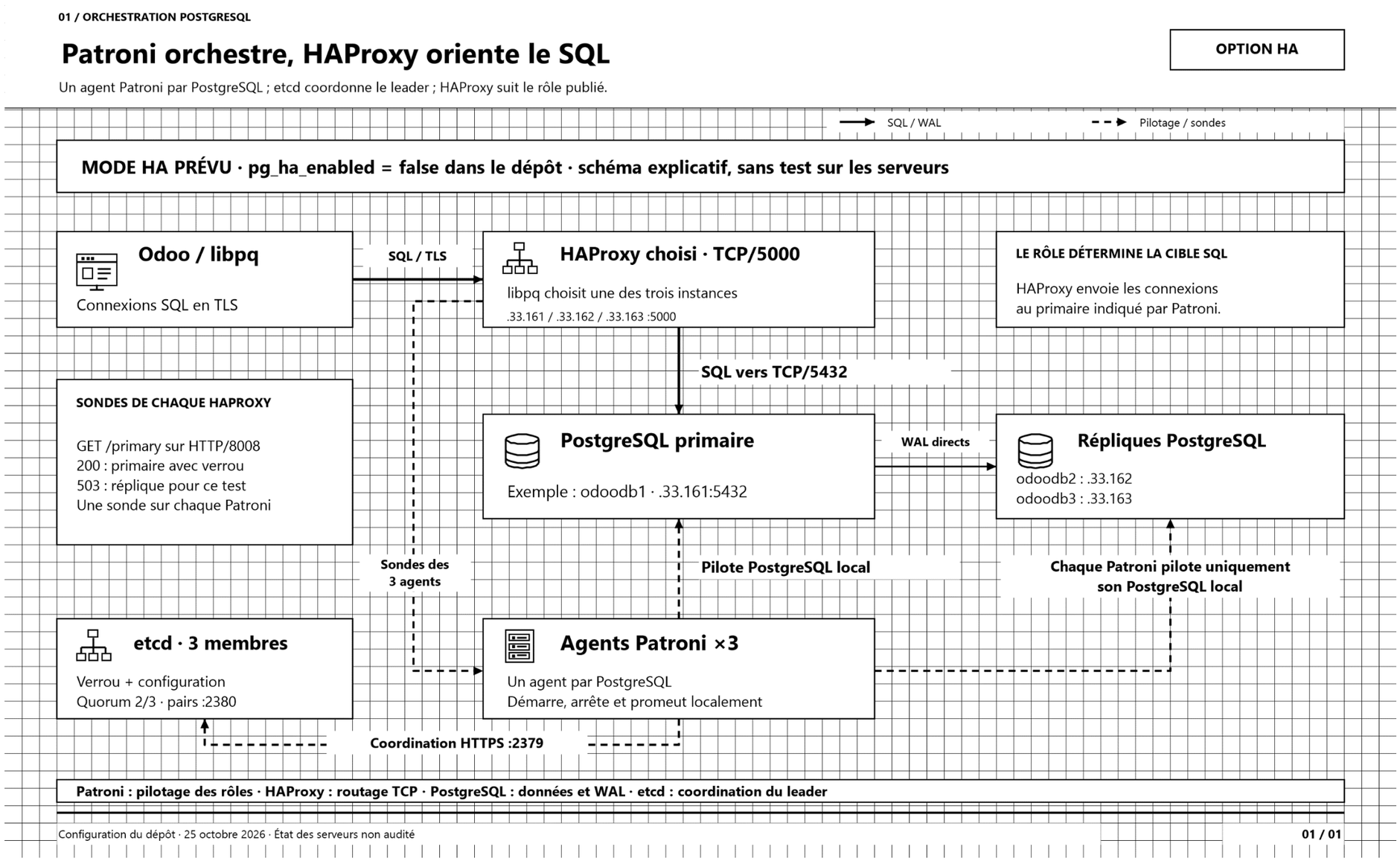
Fonctionnement : Le principe demeure valable, mais la présence d’un quorum ne signifie pas zéro perte en réplication asynchrone. Le choix d’odoodb2 dans un exemple n’établit pas une préférence permanente.

À retenir : Les figures 6 et 7 détaillent le parcours actuel ; le chapitre 5.1 précise aussi la différence entre un cluster configuré en HA et un cluster temporairement en pause.

Source : docs/schemas-architecture-ha/README.md ; rôle pg_ha | Fichier : docs/schemas-architecture-ha/architecture-actuelle-et-ha-postgresql.drawio

Figure 27 — Première orchestration Patroni / HAProxy / PostgreSQL

HISTORIQUE — statut d'activation ancien, logique toujours utile



Lecture : Les flux SQL traversent HAProxy, les WAL relient PostgreSQL aux répliques, et les pointillés représentent la supervision locale et la coordination. Un groupe de trois agents représente trois processus distincts.

Fonctionnement : Le dessin reste utile pour séparer les responsabilités des composants. Le proxy représenté est l'un des trois disponibles, pas un équipement central unique.

À retenir : La mention HA prévue et pg_ha_enabled=false appartient à la date de cette figure. La configuration actuelle true est expliquée dans l'atlas principal.

Source : docs/schemas-patroni/README.md | Fichier : docs/schemas-patroni/orchestration-patroni-haproxy-postgresql.drawio

9. Schémas textuels de la documentation

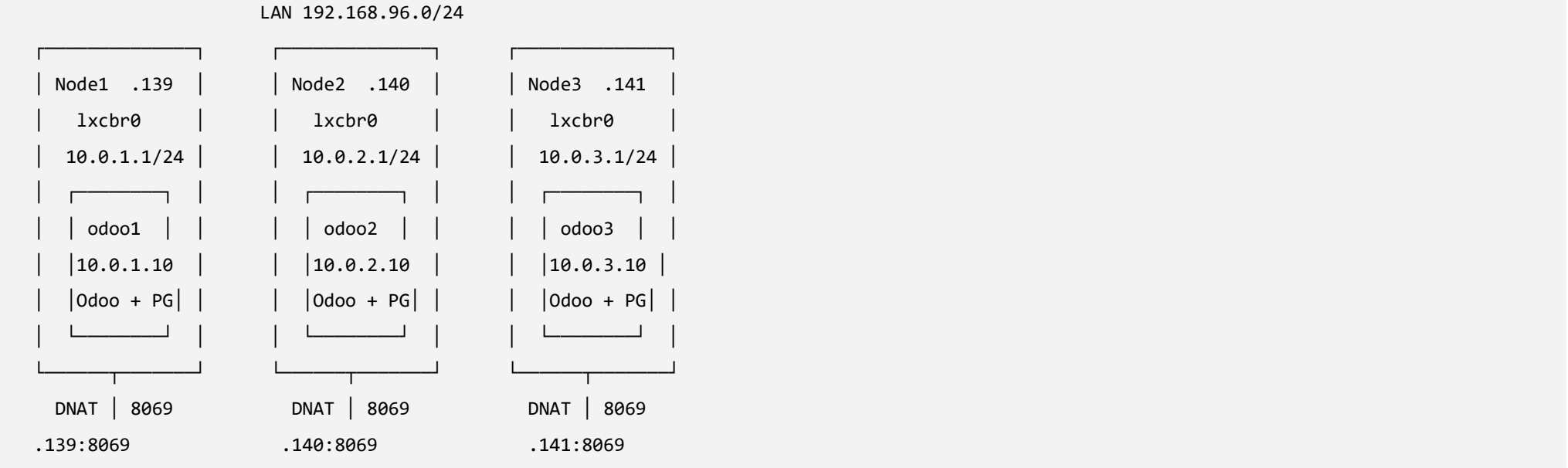
Les douze extraits suivants conservent aussi les schémas d’architecture dessinés en caractères dans les fichiers Markdown et les commentaires Ansible. Ils sont intégrés comme texte monospacé, afin de rester lisibles et modifiables dans Word.

Les anciennes adresses et les anciens volumes sont volontairement conservés dans ces extraits. Chaque page indique leur statut et leur évolution. La vue Nginx est une proposition ; elle ne prouve pas le déploiement d’une entrée web unique.

La source Mermaid architecture-actuelle.mmd représente la figure graphique 15, déjà incluse. Les fichiers SVG et draw.io correspondant aux PNG sont des variantes de format et non des architectures supplémentaires.

T01 — Première organisation en instances autonomes

HISTORIQUE — ancien laboratoire .96.x / réseaux 10.0.x

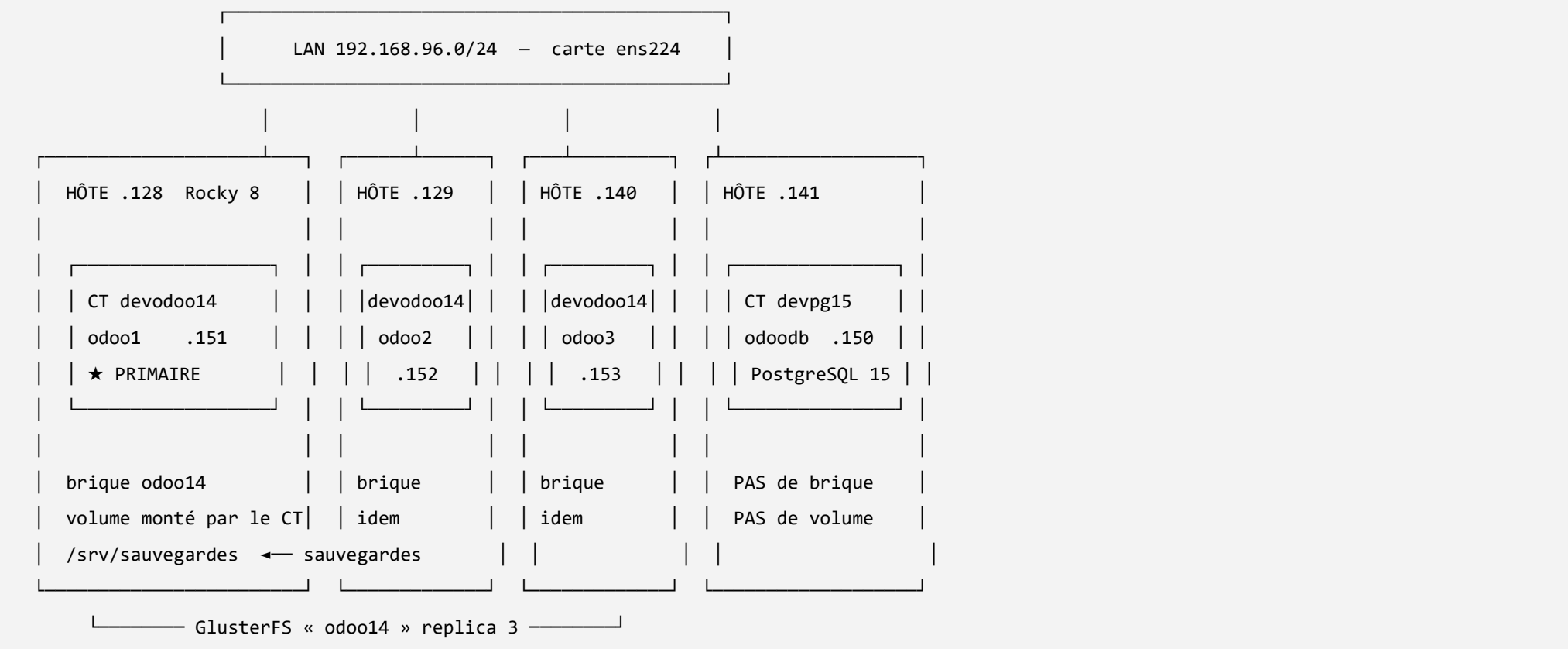


Les trois cases regroupent chacune Odoo et PostgreSQL. Cette première organisation utilisait des bases indépendantes et un accès par redirection de ports. Elle a été remplacée par trois applications partageant une base et un filestore ; les adresses de cet extrait ne sont plus les adresses de référence.

Source : README.md ; schéma textuel 1

T02 — Ancienne organisation à quatre hôtes

HISTORIQUE — ancien réseau et anciennes briques locales

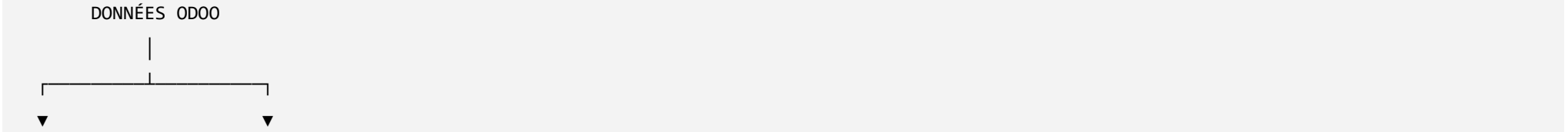


Le dessin distingue trois applications et un serveur PostgreSQL dédié, dans le réseau .96.x. Les briques GlusterFS étaient représentées sur les hôtes applicatifs. L'architecture actuelle déclare six hôtes, deux LAN et un GlusterFS externe.

Source : docs/ARCHITECTURE.md ; schéma textuel 1

T03 — Les deux familles de données Odoo

PRINCIPE ACTUEL — adressage historique



La séparation entre données SQL et contenu des pièces jointes reste essentielle. Les références .150 et replica 3 appartiennent à cet état documentaire. La restauration complète doit toujours réunir la base et le filestore cohérents.

Source : docs/ARCHITECTURE.md ; schéma textuel 2

T04 — Ancienne arborescence du volume partagé

HISTORIQUE — nom et serveurs du volume remplacés

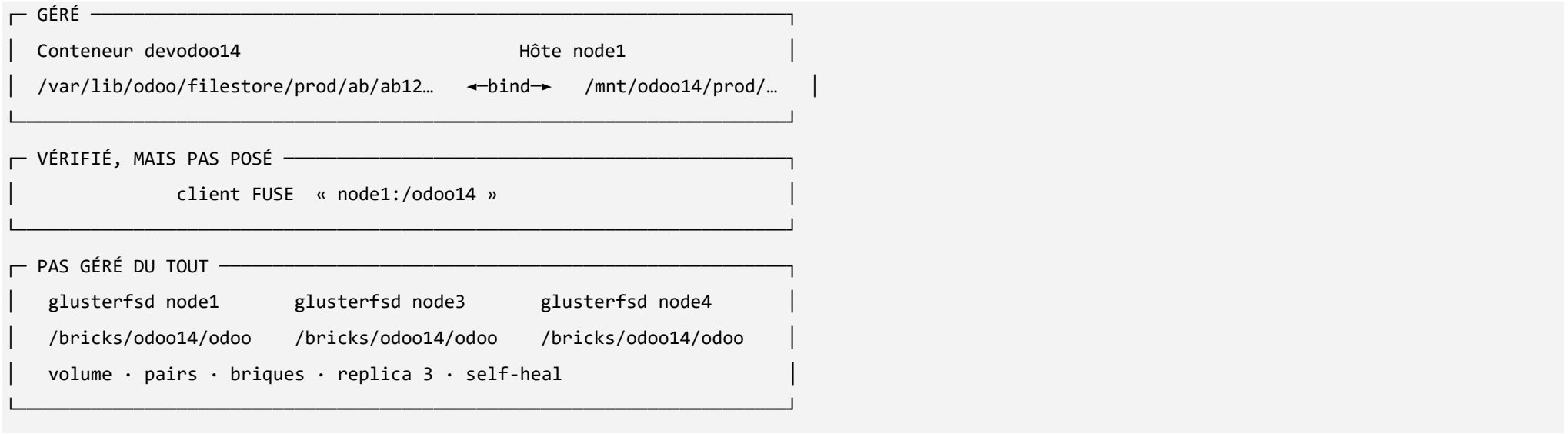
```
volume gluster « odoo14 »   replica 3, briques /bricks/odoo14/odoo sur .128 .129 .140
      │
      │  /etc/fstab DU CONTENEUR — node<n>:/odoo14 glusterfs
      │  defaults,_netdev,backup-volfile-servers=…
      ▼
/var/lib/odoo               ← le data_dir ENTIER, dans le conteneur
      ├── filestore/prod    ← là où Odoo écrit ses pièces jointes
      ├── sessions          ← partagé par construction (§3.3)
      └── addons
```

L’arbre explique qu’un montage sur le data_dir entier partage ses sous-répertoires. Le volume actuel est sre fourni par une infrastructure externe. Le code et le venv actifs restent locaux ; le répertoire addons de ce schéma n’établit pas un addon personnalisé déployé.

Source : docs/ARCHITECTURE.md ; schéma textuel 3

T05 — Ancien périmètre du montage et des briques

HISTORIQUE — périmètre depuis étendu au montage conteneur

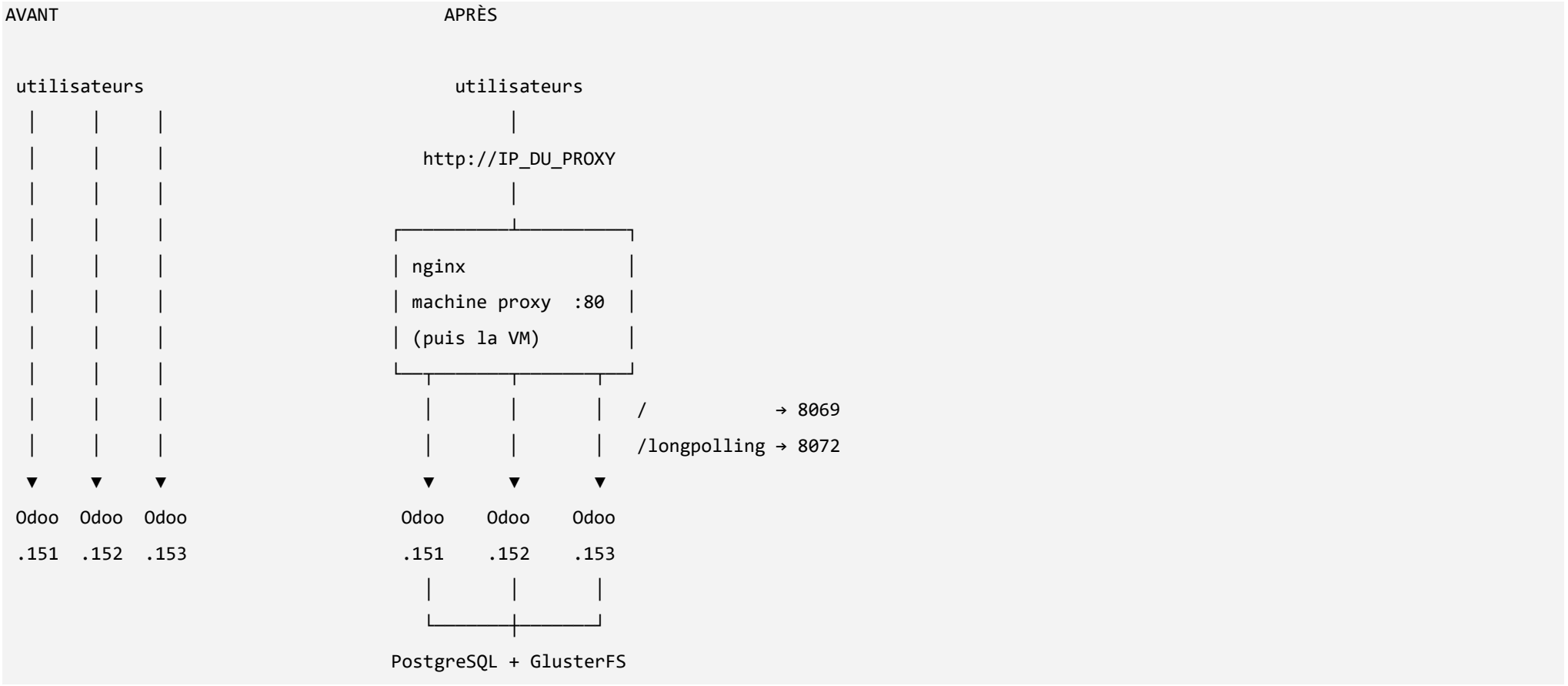


Les trois niveaux distinguent ce qui était posé, vérifié ou externe. Aujourd’hui, le parcours actif configure lui-même le montage GlusterFS dans chaque LXC Odoo. La topologie des briques externes demeure en dehors du dépôt.

Source : docs/PERIMETRE-ANSIBLE.md ; schéma textuel 1

T06 — Proposition de répartiteur HTTP Nginx

PROPOSITION — non établie comme déployée

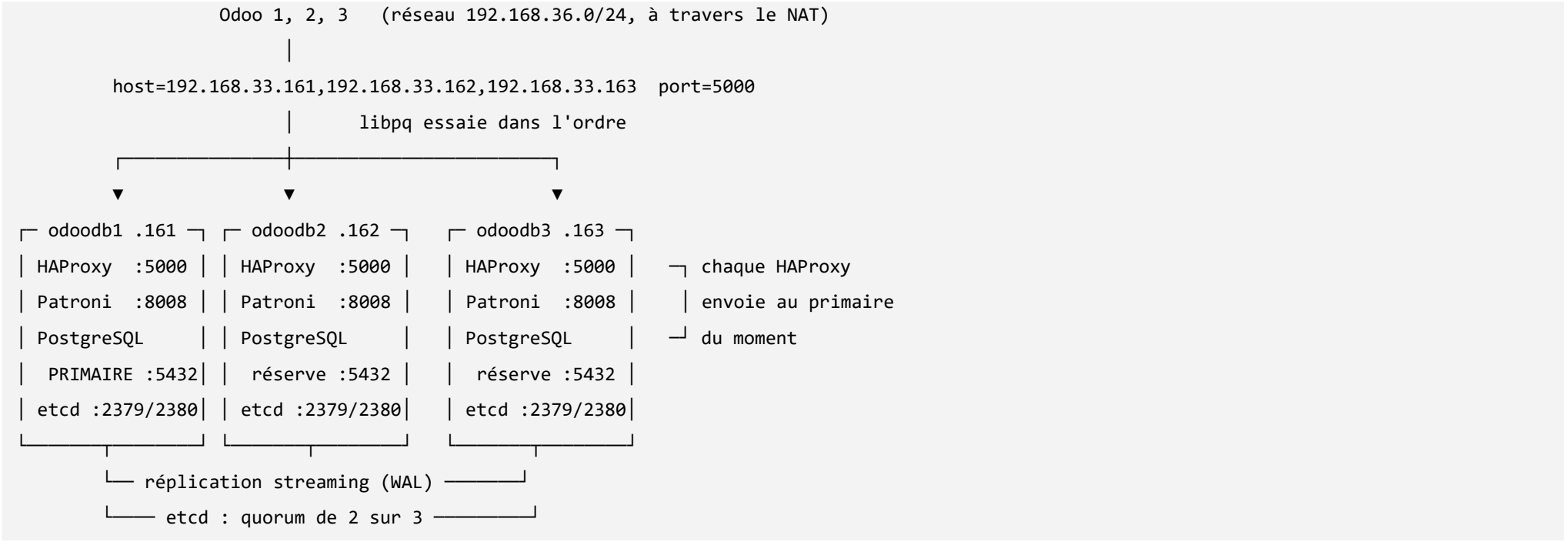


La colonne après propose une entrée web unique envoyant / vers 8069 et /longpolling vers 8072. Le fichier d'exemple utilise encore des adresses historiques. Ce répartiteur HTTP est distinct des HAProxy SQL du cluster PostgreSQL.

Source : docs/NGINX-REPARTITION-CHARGE.md ; schéma textuel 1

T07 — Vue textuelle du cluster Patroni

CONFIGURATION ACTUELLE — primaire montré à titre d'exemple



Libpq choisit l'un des trois endpoints HAProxy sur 5000. Chacun route vers le primaire annoncé par Patroni. Les membres etcd forment la coordination 2/3 ; les PostgreSQL échangent directement les WAL. Cette vue complète les planches graphiques sans constituer une nouvelle topologie.

Source : docs/POSTGRESQL-PATRONI.md ; schéma textuel 1

T08 — Comparaison lxcbr0/NAT et bridge L2

HISTORIQUE — ancien laboratoire réseau .96.x

```
AVANT (lxcbr0, NAT)                APRÈS (br0, bridge L2)


  [conteneur 10.0.3.x]               [conteneur 192.168.96.150]
         |                                    |
     [lxcbr0] -- NAT --> [ens160]          [br0] --- [ens160]
         |                     |              |          |
     (isolé)              192.168.96.139   (même L2)  LAN 192.168.96.0/24
```

Le comparatif oppose un conteneur derrière un réseau privé et une redirection NAT à un conteneur directement placé sur le LAN par un bridge. Il décrit une étape ancienne d'étude du réseau. La configuration retenue dans l'inventaire actuel est IPVLAN L2, pour partager la MAC de l'hôte sans céder sa carte ni imposer un bridge physique.

Source : docs/bridge-lxc-recapitulatif.md ; schéma textuel 1

T09 — Parcours historique de migration des pièces jointes

PROCESSUS HISTORIQUE — ligne du bind à contextualiser

```
0   préflight      horloges, montage, UID, volume à migrer
1   sauvegardes    pg_dump + tar par nœud        ← tant que rien n'a bougé
2   ARRÊT          les trois Odoo                ← aucune écriture pendant
3   UID            105/112 partout               ← un FS retient des NUMÉROS
4   bind           /mnt/odoo14 → conteneur         ← redémarre les conteneurs
5   droits         chown du filestore
5b  récupération   verse les anciens filestores  ← SINON perte silencieuse
6   migration      un seul nœud, le primaire
7   bilan          VACUUM FULL + vérification
8   redémarrage
```

Ce schéma d’enchaînement explique pourquoi sauvegardes, arrêt des écritures, identités, stockage, récupération des fichiers et contrôles doivent être ordonnés. La ligne /mnt/odoo14 vers le conteneur appartient à l’ancien montage hôte/bind. Le parcours actif importe filestore-conteneur.yaml pour le montage direct ; la migration demeure une opération avec interruption.

Source : docs/EXPLICATION-DES-PLAYBOOKS.md ; schéma textuel 1

T10 — Évolution du montage : hôte/bind vers conteneur

COMPARAISON ACTUELLE — commentaire du playbook actif

```
avant : 192.168.36.8:/sre -> /mnt/odoo (HOTE) -> bind -> /var/lib/odoo
apres : 192.168.36.8:/sre ---------------------------> /var/lib/odoo
                                                       (CONTENEUR)
```

Le volume logique est identique avant et après ; c’est le système client qui effectue le montage qui change. Le montage direct rend la dépendance au vrai volume visible dans le conteneur Odoo. Le parcours retire le bind historique et donne au LXC l’accès FUSE requis. Seul l’hôte de sauvegarde conserve un montage en lecture seule.

Source : playbooks/filestore-conteneur.yaml ; schéma textuel 1

T11 — Chaîne de stockage visée par la migration actuelle

CONFIGURATION ACTUELLE — commentaire d’architecture

```
192.168.36.8:/sre                  <- volume GlusterFS replique, HORS
      |                               projet (gluster_server/_volume)
      |  mount -t glusterfs, depuis le conteneur (play 4/8)
      v
/var/lib/odoo                      <- CONTENEURS odoo1-3 (.36.161-163)
                                      filestore/ et sessions/ COMMUNS
```

Les clients GlusterFS des conteneurs montent le volume externe sre sur leur data_dir commun. Le filestore et les sessions sont donc partagés. Cette vue confirme le montage décrit dans le playbook actif ; elle ne signifie pas que la migration est exécutée en permanence ni que toutes ses anciennes cibles statiques sont automatiquement adaptées à une bascule Patroni.

Source : playbooks/ansi_syncro.yaml ; schéma textuel 1

T12 — Chaîne historique à deux étages du rôle odoo_filestore

HISTORIQUE — commentaire du rôle de montage hôte

```
{{ gluster_server }}:/{{ gluster_volume }}   volume replique, hors projet
      |  mount -t glusterfs (fstab)        <- pose par CE role
      v
/mnt/odoo                                  sur l'HOTE Debian
      |  lxc.mount.entry (bind)
      v
/var/lib/odoo                              dans le CONTENEUR Debian
                                           (data_dir entier : filestore
                                           ET sessions sont partages)
```

Le commentaire montre l'ancien montage du volume sur /mnt/odoo dans l'hôte, puis son exposition au LXC par bind. Il reste utile pour comprendre l'origine des anciens schémas GlusterFS. Les justifications de ce rôle décrivent ce modèle antérieur ; le parcours actif assume désormais le client FUSE dans chaque conteneur.

Source : roles/ansi_odoo/odoo_filestore/tasks/main.yaml ; schéma textuel 1

10. Glossaire — application et infrastructure

Terme	Explication dans ce projet
ERP	Ensemble d’applications de gestion reliées par des données communes.
Module / addon	Application ou extension Odoo qui ajoute des objets, écrans et règles métier.
Instance Odoo	Processus applicatif ; plusieurs instances peuvent servir une même base.
Base prod	Nom de base partagé par défaut ; surcharge possible via PROSYS_ODOO_DB.
Filestore	Fichiers des pièces jointes, reliés à leurs références SQL.
data_dir	Racine des données Odoo ; ici /var/lib/odoo, partagée avec les sessions.
LXC	Conteneur Linux qui isole le système applicatif ou le système de base.
Hôte LXC	Machine qui exécute le conteneur ; son IP diffère de celle du LXC.
Venv	Environnement Python local contenant l’interpréteur utilisé et les bibliothèques de l’application.
Worker	Processus qui traite les requêtes applicatives.
Cron	Traitements Odoo planifiés ; un thread configuré sur odoo1.
IPVLAN L2	Accès réseau du conteneur avec IP propre et MAC de la carte parente partagée.

Les définitions décrivent l’emploi des termes dans les fichiers du projet.

10.1 Glossaire — disponibilité et exploitation

Terme	Explication dans ce projet
Primaire PostgreSQL	Serveur SQL qui accepte les écritures ; son emplacement peut changer.
Réplique	Copie PostgreSQL qui reçoit et rejoue les WAL du primaire.
WAL	Journal des changements PostgreSQL utilisé notamment pour la réplication.
Réplication asynchrone	Le primaire peut confirmer une transaction avant sa réception par une réplique.
DCS / etcd	Service de coordination du cluster, qui conserve verrou et configuration.
Quorum	Majorité nécessaire à la décision de coordination ; 2 membres etcd sur 3.
Failover	Promotion d'une réplique après perte du primaire, sous conditions.
Switchover	Changement de primaire demandé pour une opération planifiée.
pg_rewind	Outil de remise en cohérence d'un ancien primaire avec une timeline différente, sous préconditions.
Pause Patroni	État de maintenance suspendant la bascule automatique sans changer la variable HA du projet.
TLS / verify-full	Chiffrement SQL et vérification de l'autorité et de l'identité du serveur.
Restauration	Reconstruction d'un état cohérent à partir des sauvegardes SQL et fichiers.

10.2 Sources du projet

Les références suivantes permettent de retrouver la preuve de configuration. Les chemins sont relatifs à la racine du dépôt ; ils n'exigent pas que le rapport conserve un lien avec les fichiers pour afficher les schémas intégrés.

Référence	Fichiers ou ensembles consultés
S01	inventory.yaml ; host_vars/192.168.36.21–23.yml ; host_vars/192.168.33.21–23.yml
S02	group_vars/all/vars.yml
S03	playbooks/ansi_odoo.yaml ; ansi2-snapchot.yaml ; reseau-lxc.yaml ; ansi_postgresql_ha.yaml
S04	roles/ansi_odoo/odoo14/{defaults,tasks,meta,templates}
S05	roles/ansi_odoo/odoo_dbinit/{defaults,tasks}
S06	roles/ansi_odoo/odoo14/templates/odoo.conf.j2
S07	group_vars/odoo_containers/vars.yml ; group_vars/odoo_primary/vars.yml ; roles/ansi_odoo/odoo_uid
S08	roles/ansi_postgresql_ha/pg_ha ; docs/POSTGRESQL-PATRONI.md
S09	playbooks/filestore-conteneur.yaml ; roles/filestore-conteneur/filestore_conteneur
S10	roles/ansi_odoo/python38/{defaults,tasks}
S11	playbooks/ansi_syncro.yaml ; roles/ansi_syncro/odoo_migration
S12	playbooks/nettoyage_odoo.yaml ; roles/nettoyage_odoo/odoo_cleanup
S13	group_vars/backup_host/vars.yml ; roles/ansi_sauvegarde/odoo_backup
S14	docs/schemas-* ; docs/architecture-reseau-lxc.drawio ; docs/architecture-actuelle.{png,svg,mmd}

Les README et anciens documents ont été utilisés comme historique, puis confrontés aux valeurs et tâches actuelles. Le rôle de création reste compatible avec plusieurs familles de systèmes ; l'inventaire actuel et les conteneurs décrits sont distincts des anciens laboratoires Rocky.

10.3 Références fonctionnelles officielles Odoo 14

Ces références étayent les explications produit des chapitres 2.1 et 2.2. Elles ne constituent pas une preuve d'installation des modules dans prod. Les fonctions exactes dépendent des applications, de l'édition et du paramétrage.

O01 — CRM : pistes et opportunités

O02 — Applications de vente

O03 — Point de vente

O04 — Achats

O05 — Stock et logistique

O06 — Comptabilité et rapprochement

O07 — Ressources humaines : salariés

O07b — Ressources humaines : recrutement

O08 — Temps et prestations

O09 — Fabrication

O10 — Commerce électronique

O11 — Marketing

O12 — Assistance clients

O13 — Maintenance des équipements

O14 — Installation et éditions

10.4 Références HA et couverture des schémas

R01 — API Patroni : rôle primaire et actions

R02 — Réplication et perte possible en mode asynchrone

R03 — Watchdog et protection du primaire

R04 — pg_rewind PostgreSQL 15

R05 — Pannes et partitions etcd

Inventaire de couverture

Ensemble	Vues intégrées	Repères
Dossier Patroni / Odoo du 3 novembre	14	Figures 1 à 14
Synthèse du 16 octobre	1	Figure 15
Architecture réseau LXC sans PNG initial	1	Figure 16 — rendu exporté pour ce rapport
Schémas LXC du 18 octobre	3	Figures 17 à 19
Schémas GlusterFS du 18 octobre	3	Figures 20 à 22
Architecture et bascule du 25 octobre	4	Figures 23 à 26
Première orchestration Patroni	1	Figure 27
Schémas textuels et commentaires des sources	12	Extraits T01 à T12

Les formats PNG, SVG, draw.io et Mermaid d'une même vue ne sont pas comptés plusieurs fois. La planche des quatorze vues est un index visuel. Les images du rapport sont intégrées au DOCX ; les sources éditables restent dans le projet.

Édition du 3 novembre 2026. Fin du rapport.